國立成功大學
測量及空間資訊學系
碩士論文

基於自動編碼器與條件生成對抗網路之旅程推薦應用

Trip Recommendation Based on Autoencoder and Conditional GAN

研 究 生：賴元淯
指導教授：呂學展

中華民國一一二年七月

中文摘要

因應疫情的解封，各地開始開放觀光，許多民眾紛紛前往各地旅遊。熱門的景點能吸引大量的遊客，但是過多的人流時常造成當地交通壅塞，使環境超出容忍負荷，也帶來不好的旅遊體驗。除此之外，規劃旅遊的行程也同樣困擾許多使用者，因此已有許多論文提出各種旅遊推薦的方法，目的是在使用者提出的限制中，推薦符合使用者喜好的旅程。然而，多數的推薦方法中，都是以預測推薦旅程作為目標，很少考慮到旅遊時多樣性，失去了讓使用者探索的樂趣，也容易使遊客集中在熱門的地區。因此，本研究嘗試以不同角度切入旅遊推薦問題，旨在提出一種以多樣性為目標的旅程推薦方法，同時提供使用者輸入欲達成的條件，希望給使用者彈性的調整空間又能推薦多樣的旅程。在方法中，我們使用了基於遞迴神經網路的自動編碼器及條件生成對抗網路來生成符合使用者條件的推薦軌跡。具體來說，編碼器能將過往的軌跡資料提取出特徵，透過生成式網路的學習，我們能生成不同樣貌的特徵向量，再藉由解碼器重建成推薦軌跡。最後，我們在多個真實資料集中與其他推薦方法進行實驗，結果顯示我們的方法在多樣性方面幾乎都是最好的表現，且其他指標也都具有一定的水準。

關鍵詞:自動編碼器、遞迴神經網路、條件生成對抗網路、旅程推薦、多樣性

Abstract

As travel restrictions are gradually lifted in response to the pandemic, people have started to engage in tourism activities and travel to various destinations. Popular attractions attract a large number of tourists, but the excessive influx of visitors often leads to traffic congestion and strains the local environment, resulting in a terrible travel experience. Therefore, several researches have proposed various methods for trip recommendation with the aim of suggesting itineraries that align with user preferences and constraints. However, most of these recommendation methods focus primarily on predicting the recommended trip, often overlooking the importance of diversity in travel. Hence, we attempt to cut into the travel recommendation problem from different angles, aiming to propose a travel recommendation method that aims at diversity, and at the same time allows users to input their desired conditions. In our proposed method, we utilize an Autoencoder based on Recurrent Neural Network (RNN) and a conditional generative adversarial network (CGAN) to generate recommended trajectories that meet user-specified conditions. Specifically, the Autoencoder extracts features from historical trajectory data, and through the learning process of the generative network, we generate diverse feature vectors. These vectors are then decoded to reconstruct the recommended trajectories. Finally, we conduct experiments using multiple real-world datasets and compare our method with other recommendation approaches. The results demonstrate that our method is almost the best in terms of diversity, while also achieving satisfactory performance on other evaluation metrics.

Keyword: Autoencoder、RNN、CGAN、trip recommendation、diversity

誌謝

兩年的研究所時光很快就過去了，在這兩年間，我在 MAGIC Lab 學到了很多，也得到了很多，我不是一個擅長惜福與表達的人，但是身在這間實驗室卻讓我感到很幸福，也讓我想一直珍惜在這裡的時光。我想先感謝呂學展老師，老師總是以朋友的身分與我們相處，讓我在做研究時不會感受到輩分的壓力，老師也很常給我建設性的建議，並且指導我做研究的心態與思路。會講幹話的老師不多，但學展老師正好就是那屈指可數的其中一個，讓我的碩士生涯增添不少歡笑。Perfe 也是會講幹話的人，謝謝他常常為已經很冷的實驗室降溫，也謝謝桂華、阿梅慷慨地請我們吃零食還有陪我們玩，你們的研究意見給了我很大的啟發。謝謝佑儒老哥在工作閒暇之餘還願意與我們五排，也謝謝宜均、姿穎參與我的碩士生活。感謝我的好夥伴貫平、韋智跟著我一起在成大吵吵鬧鬧，陪我度過實驗室的種種蠢事。謝謝沈晴、小八陪我們吃晚餐，也感謝奕萱在我們需要時幫助我們修訂論文，這真的為我們省下很多時間。最後謝謝黃科瑋老師、陳奕中老師、蘇家輝老師喜歡我的研究，也給我很多有價值的修改建議。

另外，也有許多實驗室以外的人給我很多幫助，感謝尚峰、東華、諺恩、品翰給了我很多口試方面的改善方向，讓我的口試報告更為順暢，也在我有困難時給我幫助。還有感謝晟安一直在社群上與我互動，傾聽我的煩惱。當然還要謝謝系辦的各位幫忙各種繁雜的行政流程，作為我們在學校的依靠。感謝我的家人們在我需要時，提供我一個溫暖又安心的避風港可以回去依賴。最後，我想謝謝自己，謝謝元淯在這兩年來撐過口試、meeting、報告、疫情等許多鳥事，還依然能保有初衷，笑著面對往後的困難。

Content

List of Tables

List of Figures

Chapter 1 Introduction

1.1 Background

With the gradual easing of the pandemic in recent months, many countries have started reopening their tourism borders, leading to a recovery in the tourism industry and its related sectors. During this period, revenge travel has emerged in many regions, where people are flocking to various attractions in order to compensate for their inability to travel during the quarantine period. This type of tourism has brought significant tourist flows to local attractions and generated substantial revenue for the tourism related industries. According to United Nations World Tourism Organization statistics, global tourist arrivals in 2023 are projected to reach 95% of pre-pandemic levels, surpassing the 63% recorded in 2022 [45]. Moreover, as reported by The Economist, global airlines are expected to generate a net profit of $9.8 billion in 2023, and ticket prices are increasing at a faster pace than inflation [46]. These figures indicate a rapid increase in people's travel demand, surpassing pre-pandemic levels. Therefore, trip recommendations can play a crucial role at this time. By recommending suitable trip trajectories, these systems can help reduce the opportunity cost of planning trips and provide users with personalized recommendations based on their preferences, assisting them in planning their journeys. This phenomenon highlights the significant research value and significance of travel recommendation studies, as they offer various topics and objectives for further exploration and development.

1.2 Motivation

Traveling can broaden horizons and provide a sense of relaxation and enjoyment. However, terrible travel experiences can dampen the mood and hinder the opportunity for exploration. With the recent surge in travel activities, known as revenge travel, popular places have witnessed an influx of visitors, resulting in issues such as overcrowding, environmental degradation, and traffic congestion. This phenomenon is not conducive to the sustainable development of the tourism industry. In my opinion, it is crucial for trip recommendations to offer diverse options rather than solely promoting the most popular or user-preferred places. In this way, it can not only disperse the influence of crowds but also allow users to explore the characteristics of other places and drive sightseeing in different regions. Existing studies have considered the popularity of attractions in travel recommendations [17][19][21][32], but the potential consequences of recommending heavily visited attractions have not been addressed. By prioritizing diversity in recommendations, the concentration of tourists in specific areas can be alleviated, and travelers can be encouraged to explore lesser-known attractions with unique characteristics. While some approaches prioritize recommendation scores [10][30][31], or employ deep learning techniques for prediction-based recommendations [4][11][18][39], the aspect of diversity is often neglected. Therefore, we believe that incorporating diversity as a primary objective in travel recommendations holds significant research value and merits further investigation.

1.3 Research Approach

To address the aforementioned issues, we aim to recommend diverse trip

trajectories as our objective. In this thesis, we adopt popular deep learning methods, specifically generative adversarial networks (GANs), to achieve diversified recommendations. Firstly, we assume that the distribution of historical trip categories within the dataset represents users' preferences towards these categories. To facilitate deep learning, we perform data augmentation to ensure an adequate amount of training data based on this distribution. Next, we modify the architecture of the Autoencoder and incorporate the Long Short-Term Memory (LSTM) units, belonging to one type of the RNN, with Attention mechanisms. This enables the Encoder to extract low-dimensional features from trajectory data, while the Decoder reconstructs the original trajectories based on these features. Furthermore, we employ conditional GAN (CGAN) to generate low-dimensional features, considering factors such as trip length and trip group type as conditions. These low-dimensional features generated by CGAN have a certain degree of randomness and can produce diverse outputs. Finally, we input the low-dimensional features generated by CGAN according to the conditions into the Decoder to reconstruct the trajectory, and we can get a diverse trip that meets the conditions. In the comprehensive experimental results, our method demonstrates the highest diversity in most datasets and also exhibits the best accuracy in some experiments.

1.4 Contribution

1. We propose a novel approach for augmenting trip trajectory data, which preserves the original data's category distribution and allows for controlling the trajectory length.
2. We adopt a novel Autoencoder architecture that can encode trip trajectories into low-dimensional features and reconstruct the trajectories while incorporating

starting point conditions to generate specific trajectories.

3. We modify the CGAN architecture to generate low-dimensional trajectory features, which can generate diverse outputs based on random noise vectors and multiple conditions.
4. We employ multiple evaluation metrics to assess the recommended trip trajectories, considering aspects such as diversity, accuracy, redundancy, and stability of trip recommendations.
5. We conduct experiments using popular check-in datasets, and the results demonstrate that our method achieves superior diversity compared to the comparison methods in most cases.

1.5 Organization

The chapter arrangement is as follows. In Chapter 2, we provide a literature review on travel recommendations, focusing on different features and methods used in the field. Chapter 3 presents the symbols and problem definitions that will be utilized in our proposed method. Chapter 4 explains the overall process and functioning of our method. Subsequently, we evaluate the performance of our method and compare it with other travel recommendation approaches in Chapter 5. Finally, Chapter 6 summarizes our findings from the experiments and discusses future works for research.

Chapter 2 Related Work

In this chapter, we will discuss and review the past literature regarding travel recommendation and prediction. By referencing the knowledge and experiences of various scholars, we analyze the advantages and disadvantages of different methods and objectives. Typically, before recommending and predicting the trips, a feature extraction process is carried out to extract useful information from a large amount of data. The quality of this information greatly affects the outcomes of the methods. In the methods proposed in these literatures, some scholars employ heuristic algorithms with multiple constraints for solving the problem, while others improve collaborative filtering methods for recommendation. In recent years, research in deep learning has rapidly advanced, and many related packages and tools have emerged. This development has not only led to the adoption of deep learning methods in various fields to achieve research objectives but also lowered the threshold for application due to the ease of use of these tools and advancements in hardware. More and more studies are using deep learning models to process data and extract features. These studies incorporate different spatial and temporal factors as well as user preferences, thereby not only addressing the cold-start problem that arises from insufficient data in traditional recommendation systems but also improving overall performance.

Summarizing the recent travel recommendation literature, it can be observed that the research process generally involves extracting spatial-temporal features and user preferences from the data and inputting them into the proposed methods for solving the solutions or training in the models. Then, they will incorporate constraints, budgets, and other conditions to achieve experimental design objectives and implement trip recommendations. According to literature review, we divide these

studies into three topics: feature extraction factors, traditional trip recommendation, and deep learning trip recommendation.

2.1 Feature Extraction Factors

Through the organization in Table 2-1, it can be observed that feature factors are mainly divided into three types: temporal factors, spatial factors, and user preferences. Using different types of features as inputs to the methods will affect the characteristics of the resulting output.

Table 2-1 The literature of different feature extraction factors

Extraction Literature	Temporal Factors				Spatial Factors				User Preferences		
	Weather	Time	Duration	Contextual Feature	Special Feature	Popularity	Location Text	Category	Visual Feature	Similarity	Community Information
Memon *et al. [26]*	✓	✓		✓	✓					✓	✓
Wang *et al.* [35]		✓		✓	✓		✓	✓	✓	✓	
Lim *et al.* [21]		✓	✓		✓	✓		✓			✓
Dietz *et al.* [6]		✓	✓		✓			✓		✓	
Jiang *et al.* [19]		✓				✓		✓	✓	✓	✓
Sun *et al.* [29]	✓	✓		✓							
Figueredo *et al.* [7]							✓		✓		
Zhao *et al.* [43]			✓			✓			✓		
Sarkar *et al.* [32]		✓	✓			✓		✓		✓	
He *et al.* [17]				✓	✓	✓					
Abbasi-Moud *et al.* [1]	✓	✓		✓	✓		✓				✓
Jiang *et al.* [20]			✓	✓	✓			✓			
Luan *et al.* [23]		✓			✓		✓	✓			
Chen *et al.* [2]			✓			✓		✓			✓

First, in the research of trip recommendation, temporal features are essential elements. In addition to using time directly as input features, other factors with

temporal characteristics can also be considered, such as weather information and duration of stay [1][2][6][20][21][26][29][32][43]. The addition of these temporal features enriches the understanding and predictive capabilities of recommendation methods for user travel behavior. When it comes to understanding travel behavior, it is necessary to mention contextual feature, which is also relatively important temporal factors. This feature represents the relationships and connectivity between points of interest (POIs) within travel trajectories. Due to its significant impact on trajectory recommendation and prediction, as well as the widespread use of deep learning methods that can better capture features, many studies extract contextual information in their methods [1][17][20][26][29][35]. He *et al.* emphasized the importance of understanding contextual factors in the recommendation, including the popularity of POIs, relevant POIs in the current trip, user preferences, and other factors that influence the user's next decision [17]. Jiang *et al.* further incorporated the Attention mechanism from deep learning method to capture the relationships and contextual information between POIs, aiming to generate travel journeys that align with human mobility patterns [20].

A POI refers to a location or area that carries special significance, such as buildings, parks, scenic areas, and more. These POIs have different characteristics, including textual information, POI categories, geographical locations, and various other details related to the POI. Lim *et al.* proposed an algorithm called PersTour, which utilized the popularity of POIs and user preferences for personalized trip recommendations. It also considered constraints such as time, starting point, and destination. PersTour has become one of the commonly used baselines [21]. In addition to popularity and spatial features, many studies also incorporated textual information and category data of POIs into their methods to enrich feature

information and provide more robust POI features [1][2][6][7][19][20][21][23][32][35]. Luan *et al.* argued that the estimated scores of POIs based on user preferences or historical visit counts were not evenly distributed. Recommendation systems are likely to recommend similar POIs, while users have diverse preferences for POI categories. Therefore, considering travel diversity and relevance, they proposed a hierarchical POI category similarity calculation method. On the one hand, this approach aims to balance diversity and relevance, and on the other hand, it obtains usable recommended itineraries within a limited time [23]. Chen *et al.* started from the number of tourists and considered the preferences of group members in group trips when recommending the entire travel itinerary. In their proposed framework, they utilized an Attention mechanism to simultaneously learn the POI categories and textual information [2].

In addition to the features of POI content, there were some studies that collect information regarding user preferences, which are often found in media data such as photos, posts, and travelogues. By analyzing these texts, insights into social relationships among users and community information can be obtained [1][2][19][21][26]. Jiang *et al.* implemented trip recommendations by utilizing various user preference features. They not only collected diverse social media data but also optimized the recommendation method by considering the similarity between routes and users [19]. In many studies, videos and photos have been shown to provide rich hidden information, whether for POI scene recognition or understanding user preferences. Numerous studies have presented practical use cases of utilizing videos and photos [7][19][35][43]. However, categorizing these preferences according to individual preferences for recommendation purposes is not always practical. Therefore, some researches had used clustering to group user preferences or features.

By employing clustering mechanisms, individual attributes can be well allocated to similar groups [1][6][32]. Sarkar and Majumder improve on previous methods of recommending multiple trips for a single user and instead focus on recommending multiple trips for a group of users. They proposed the gtour algorithm, which considers the interests of multiple users within a group, assigns users with similar interests to the same cluster, and incorporates statistical strategies from game theory to evaluate the relationships among all participants [32].

In summarizing the feature extraction factors used in the aforementioned literature, we observe that travel trajectories consist of multiple ordered POIs, indicating the importance of temporal factors. Apart from the temporal characteristics, contextual features are also key considerations. They aid in understanding the sequential characteristics of POIs within a single trip and also provide the relationships and connectivity between POIs in the trajectory. Moreover, location features are essential factors. The influence of geographical location can be directly manifested in spatial features. In recent research, location texts and categories have played significant roles as well. They represent the information and distinctive attributes associated with each POI. Rather than relying on visitation numbers or popularity as the basis for recommendations, greater emphasis is placed on personalized itinerary creation based on POI reviews and category descriptions. Lastly, user preference factors serve as supportive information. They can be used to establish travel recommendations that cater to the general public by considering the information of multiple users. Additionally, personalized trip recommendations can be generated based on individual preference features, similar users, and community information. However, the more detailed the data, the more difficult it is to obtain, and the severe lack of data volume makes it challenging to utilize this feature factor.

2.2 Traditional Trip Recommendation

In previous research on travel recommendations, collaborative filtering techniques, widely used in personalized recommendations, are often incorporated. For example, similar user preferences can be used as a basis to predict POIs that a user might like. Alternatively, based on similar features of POIs, recommendations can be made to users who prefer those specific features, suggesting similar POIs to them. Some scholars also viewed travel recommendations as an optimization problem, seeking the best solution that minimizes costs and maximizes objective scores from a large number of POIs, while considering constraints such as time, money, and the length of trip to reduce computation time. We have summarized some literature based on constraints, objectives, and methods, as shown in Table 2-2.

Table 2-2 The literature of traditional trip recommendation

Traditional Trip Recommendation Literature	Constraints			Objectives			Methods					
	Time	Distance	Length	Trajectory	POI	Diversity	Collaborative filtering	Heuristic	Self-designed	Evolutionary	Greedy	Matrix Factorization
Liu *et al.* [24]					✓							✓
Zhang *et al.* [40]	✓			✓		✓	✓	✓				✓
Wen *et al.* [34]				✓		✓					✓	
Sun *et al.* [30]			✓	✓	✓				✓			✓
Lim *et al.* [22]	✓			✓					✓			
Gao *et al.* [8]					✓				✓			✓
Han *et al.* [15]					✓							✓
Gu *et al.* [10]	✓			✓				✓			✓	
Yochum *et al.* [36]	✓		✓	✓						✓		
Sarkar *et al.* [31]		✓		✓					✓	✓		
Noorian *et al.* [27]					✓		✓		✓			

Table 2-2 presents a comparison of the method techniques used in traditional

travel recommendation approaches. In addition to the commonly used collaborative filtering, there are studies that employ custom-designed methods and classic solving algorithms such as greedy algorithms. Each of these methods has different objectives. In the following, we will discuss these non-deep learning methods in the literature.

Liu *et al.* proposed a new recommendation method that incorporates two levels of geographic neighborhood features into the latent features of users and locations [24]. Zhang *et al.* developed a method based on multiple constraints such as the user's time budget, visitability of POIs, and diversity of POIs. They aimed to search for a POI sequence that maximizes user satisfaction [40]. Similarly, Wen *et al.* employed a score-based search approach without considering constraints such as time and space. They introduced a Keyword-aware Representative Travel Route framework to describe the relevance between keywords and trajectories, and used keywords to search for matching routes [34]. Sun and Lee clustered social network photos to identify landmarks. They then linked these landmarks with users based on the topic tags associated with users' posts and recommended popular and attractive landmarks based on similar user preferences [30]. Lim *et al.* proposed an improved method based on PersTour. They incorporated an adaptive weighting method to better adjust the parameters within the approach [22]. Gao *et al.* emphasized the importance of social information and proposed a new recommendation model that considers factors such as time, social information, and interrelationships between POIs [8]. Han *et al.* presented a travel attraction hybrid recommendation model that integrates time, visual, and spatial embeddings. This model mitigates the data sparsity issue in collaborative filtering methods and eliminates the possibility of misleading the recommendation model [15].

Unlike previous studies that focused on the popularity of POIs, Gu *et al.*

introduced a personalized recommendation approach by using the popularity of routes. They incorporated a gravity model to estimate route scores and identify attractive routes. By employing a greedy algorithm considering user experience and time constraints, they obtained an approximate solution that maximizes the recommendation effectiveness [10]. Yochum *et al.* proposed a new method based on an adaptive genetic algorithm to solve the travel recommendation problem. They assigned different weights to factors such as the popularity of POIs, user interests, and duration of stay at POIs, catering to the multiple preferences of users [36]. Sarkar and Majumder introduced the PWP algorithm, which utilizes a genetic algorithm to maximize tourist interests and minimize travel costs. It also allows for the recommendation of new POIs that users have not visited before [31]. Noorian *et al.* proposed a hybrid approach that combines personalized recommendations from multiple recommendation systems. By adjusting the parameters, they achieve a balance between different methods. They also incorporate context information and modeling, considering sequential features [27].

Based on the discussions and analysis above, we have observed that while these methods yield good solutions and recommendations, they also require significant computational time. Moreover, if the conditions change, they need to be recalculated, which may not be convenient in practical usage. In terms of constraints, travel restrictions based on a fixed budget and time may not align well with real-world scenarios. The costs of POIs in reality can vary over time, and the facilities and crowds at POIs can also lead to time exceeding the set constraints. However, limitations on the length and distance of POIs are more likely to be applicable. Users can choose not to visit redundant or distant recommended POIs, thus providing more acceptable and stable recommendations. On the objectives side, most studies focus

on recommending itineraries consisting of multiple POIs as the primary goal. Instead of recommending a single POI, users often prefer to explore different itineraries and attractions during their travels. Furthermore, we have observed that only a few pieces of literature consider recommendation diversity. Most recommendations aim at predicting users' past trips, less recommending different types of POIs from the users' perspective.

2.3 Deep Learning Trip Recommendation

In recent years, advancements in hardware technology and updates in deep learning methods have significantly reduced the conditions and thresholds for applying deep learning techniques to problem-solving. As a result, an increasing number of studies in travel recommendation have been utilizing this technology to achieve higher accuracy. Researchers have also been exploring different network architectures to improve performance and uncover deeper travel features. Common examples include Multilayer Perceptron, Convolutional Neural Networks (CNN) for extracting image features, Recurrent Neural Networks (RNN), and Generative Adversarial Networks (GAN). These methods have brought about more diverse developments and research directions in the trip recommendation field. Table 2-3 provides a compilation of literature on trip recommendation that applies deep learning methods, each with its own distinct objectives and constraints.

Cepeda-Pacheco *et al.* employed a Multilayer Perceptron (MLP) method by stacking multiple layers of neurons as the output for trip recommendation [3]. Zhang *et al.* inputted photos of the Beijing area into a pre-trained CNN for scene recognition. By analyzing the photo's time, geographic coordinates, text labels, and the scene information extracted using CNN for visual content, they examined the travel

behavior of tourists in the Beijing area [38]. Duan *et al.* further utilized the convolutional properties of CNN to transform the user's interest vector into a matrix form. They employed CNN's convolutional operations to extract high-dimensional features from the matrix [5]. Similarly using a CNN architecture, Zhao *et al.* focused on predicting the user's next destination. They employed an Attention mechanism to focus on important spatiotemporal feature vectors and incorporated them into CNN for scoring. The output with the highest probability was considered the predicted destination [44].

Table 2-3 The literature of deep learning trip recommendation

Deep Learning Trip Recommendation Literature	Constraints		Objectives			Methods					
	Time	Length	Trajectory	POI	Diversity	MLP	RNN	CNN	Attention	Autoencoder	GAN
Cepeda-Pacheco *et al.* [3]				✓		✓					
Zhang *et al.* [38]								✓			
Duan *et al.* [5]	✓		✓					✓			
Zhao *et al.* [44]								✓	✓		
Zhou *et al.* [42]			✓				✓				
Gao *et al.* [12]		✓	✓		✓		✓				
Huang *et al.* [16]				✓			✓		✓		
Huang *et al.* [18]	✓		✓	✓		✓	✓		✓		
Chen *et al.* [4]	✓		✓							✓	
Zhou *et al.* [41]			✓				✓		✓	✓	
Gao *et al.* [13]						✓	✓		✓	✓	✓
Gao *et al.* [14]		✓	✓				✓			✓	✓
Gao *et al.* [11]	✓	✓	✓			✓	✓		✓	✓	
Zhao *et al.* [39]		✓	✓			✓	✓		✓	✓	✓

However, CNN is designed for image data and is not effective in learning sequential data features. Therefore, Zhou *et al.* introduced the concept of self-

supervision and used RNN as the basic unit of their model. RNN considers the previous output as the current input, allowing the collection of spatio-temporal context information at each position in the trajectory. This enables the incorporation of sequential features from the time series [42]. Gao *et al.* employed a self-supervised framework to train POI and trajectory features for trip recommendation. They proposed a random walk strategy to enrich user query demands and increase travel diversity. Additionally, they used negative sampling for contrastive learning to enhance the richness of the dataset and improve the training results [12]. Huang *et al.* treated the prediction of the user's next destination as the objective. They considered the order of user's POI check-ins and their contextual information as key factors in analyzing user behavior. Thus, they utilized the LSTM model, a type of RNN, as the foundation. LSTM can better capture long-term and short-term sequential features, outperforming traditional RNN. Furthermore, they incorporated an Attention mechanism. By adding the Attention mechanism, the LSTM can selectively focus on sequence points that contribute more to the current state, effectively modeling spatio-temporal context information [16].

In previous studies on trip recommendation, the focus was usually on either recommending itineraries or the next POI separately. However, Huang *et al.* achieved both objectives using a single method. In addition to LSTM, Attention, and MLP, they incorporated the Beam search to assist the model in recommending POIs as itineraries and incorporating conditions for necessary visits, thereby achieving flexible and diverse trip recommendations [18]. In contrast to previous research that utilized RNN and Attention networks, Chen *et al.* focused on extracting textual information from POIs. They introduced an autoencoder method that can learn to compress and decompress data, altering the output style after decompression. By doing so,

unnecessary words and noise can be removed from the cluttered text, resulting in cleaner text feature vectors [4]. Zhou *et al.* proposed a semi-supervised trajectory-to-trajectory learning model to capture latent semantics in user mobility patterns [41]. Gao *et al.* employed semi-supervised learning methods to address trajectory classification problems. In addition to utilizing RNN and attention mechanisms as part of the autoencoder structure, they extensively employed GAN. GAN consists of a generator and a discriminator. This generator not only produces realistic data but also provides useful and structured latent features for trajectory classification models, improving overall classification performance [13]. In another work by Gao *et al.*, they employed a similar framework but removed the Attention mechanism. They trained the latent features compressed by the autoencoder together with GAN, generating latent vectors similar yet distinct from previous trajectories. The trajectories are then reconstructed using a decoder, achieving the goal of recommendation itineraries [14]. In recent research, Gao *et al.* argued that previous studies viewed trip recommendation as a simple directed off-road problem or maximized user preferences through multiple rounds of iterations, without effectively considering the spatio-temporal correlation between itineraries and POIs. Therefore, they proposed a representation learning framework based on spatio-temporal graphs and incorporated a dual-grained learning module to enhance the learning of human mobility patterns [11]. Zhao *et al.* emphasized the lack of capability in capturing the correlation between POIs in existing methods. They introduced an adversarial learning-based graph contextualized trip recommendation model to capture global-level POI embeddings. [39].

Based on the above review, we can observe that most of the current research on trip recommendation utilizing deep learning methods has applied the powerful

computational and learning capabilities of this technology. They have performed more complex and detailed feature learning on past itineraries, POI data, and user texts, enhancing the spatio-temporal relationships of POIs and contextual information of itineraries. Based on these foundations, they have learned human mobility patterns, addressing issues such as the time-consuming nature, limited usage, and monotony of traditional trip recommendation methods. However, there are still many areas where these deep learning methods can be further improved, such as the diverse generation capabilities of GAN. Many studies tend to focus on the accuracy of POI predictions and time complexity, paying less attention to evaluating the diversity of recommendations generated by the methods. Therefore, diversity evaluation is a research objective worth exploring. Additionally, the inclusion of diverse attributes in travel data and the utilization of augmented data to enhance training outcomes have further increased the research potential of applying deep learning methods for diverse analysis.

Chapter 3 Problem Statement

In this chapter, we formally define the problem that this study aims to address. Our topic is on recommending diverse trip itineraries based on queries provided by users. The symbols used in the recommendation and preprocessing are listed in Table 3-1.

Table 3-1 Notations about trip recommendation

Notation	Description
p, P, P_j	POI, POI set, POI set belonging to c_j
c, C	POI category, POI category set
f_p	The number of occurrences of p
tra	Trajectory
u	User
q	Query
l	Trajectory length
g	Trip group type
ϕ_{ij}	Generation probability of POI p_i in c_j
$vtra$	Augmented trajectory
ρ	The number of augmented data items of each length
d	Dimension
η	Encoded vector after Encoder output
r	Random vector

Definition 1. POI: A POI p represents a store, landmark or institution in dataset, which is an element that makes up a trajectory. Each element p belongs to only one category c and does not have more than one category attribute simultaneously. P denote a set of p and each $p \in P$ has its own ID $p.id$. P_j represents the set of all POIs that belong to category j.

Definition 2. POI Category: Let c represents the category of a POI p, which includes categories such as restaurants, entertainment facilities, stations, and so on. A set of categories C contains non-duplicate categories c. $|C|$denotes the number of

unique categories in the set C.

Definition 3. The frequency of a POI: f_{p_i} represents the total number of times p_i appears in all trajectories. It can be obtained by counting the occurrences of each POI $p \in P$ in all trajectories.

Definition 4. Trajectory: A trajectory tra represents the past travel history of a user u and consists of a sequence of p in order. Let $tra = \{p_1, p_2, \dots, p_N\}$ represent a trip trajectory, where N denotes the length of tra.

Definition 5. Query: A query $q = (p_s, l, g)$ represents the travel starting conditions provided by the user. p_S is the starting point POI, l is the length of the trip, and g represents the trip group type.

Problem Definition: For a given query q provided by a user, the goal is to recommend a diverse trip that meets the specified conditions in q.

Chapter 4 Proposed Method

In this chapter, we will provide a detailed description of our proposed method. We divide our method into four sections. Chapter 4.1 describes the overall architecture and the general workflow. Then, in Chapter 4.2, we explain how data augmentation is performed within the architecture. Chapter 4.3 elaborates on how the model extracts features from trajectories. In Chapter 4.4, we discuss the architecture of the trip recommendation model and explain how the proposed method is applied for recommendation purposes.

4.1 Framework

Our method's architecture is illustrated in Figure 4-1. The overall architecture consists of four components: Data Augmentation (in blue color), Trajectory Feature Extraction Model (in green color), Recommendation Generation Model (in red color), and Trip Recommendation Process (in yellow color). At the beginning of the method, the first step is to augment the trajectory data from the original dataset to meet the requirements of the training model. We will discuss how we expand the data using the original dataset in Chapter 4.2. Once we have the desired amount of data, it will be fed into the trajectory feature extraction model for training, learning to extract trajectory features. The architecture of the trajectory feature extraction model will be described in detail in Chapter 4.3. These extracted data will then serve as the training data input for the recommendation generation model. After the training process is completed, we can input a query q into the trained model, and the model will generate a recommended trip. The architecture of the recommendation generation model and the recommendation process will be introduced in Chapter 4.4.

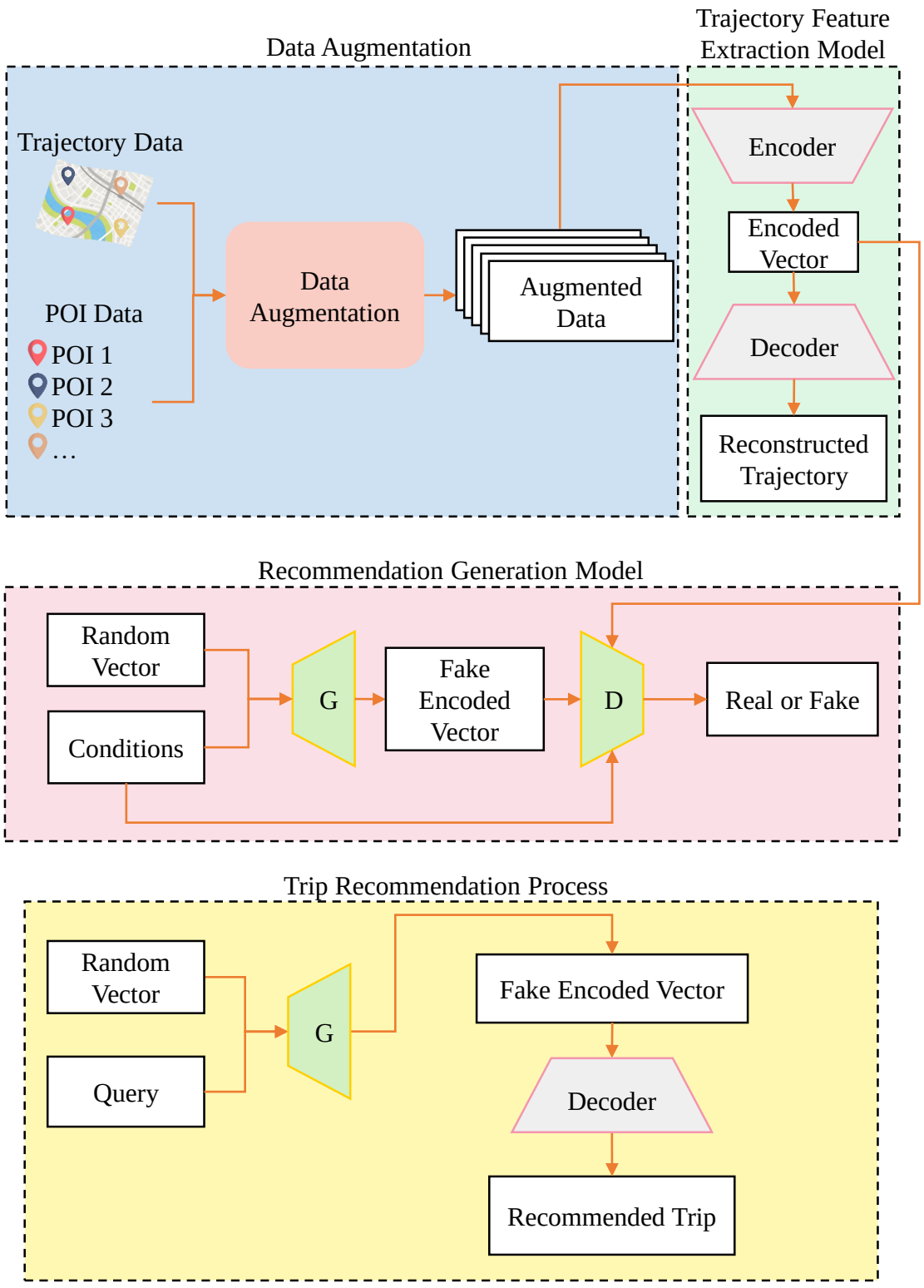


Figure 4-1 The framework of our research

4.2 Data Augmentation

In this chapter, we provide a detailed explanation of how to preprocess and augment the dataset used in our study. Since most of the datasets used in trip recommendations in recent years are check-in datasets and most of these datasets are fragmented and non-sequential. We need to extract continuous and trajectory-forming data from them. Therefore, we adopt the filtering method used in [12] for data preprocessing. Specifically, we only consider check-in data with a continuous check-in length of three or more, which we treat as one travel trajectory. After processing these trajectory data, most of the scattered check-in data is filtered out, and the remaining data is organized into trajectories tra based on user and time order, where each trajectory consists of p. However, the number of these trajectory data is insufficient for deep learning methods. Thus, we employ data augmentation techniques on these trajectory data to augment the dataset and attain the desired volume of data. The detailed steps are as follows:

I. Starting Point: Each $p \in P$ in the dataset takes turns serving as the starting point.
II. Collecting POIs: For each trajectory tra, we gather all POIs p that appear in it. We then calculate the frequency of occurrence for each category c based on all the collected p.
III. Selecting Category: We choose a POI category c_j based on the frequencies of different $c \in C$ appearing.
IV. Selecting POI: Based on the selected c_j, we calculate the probability of selecting a generated POI from the POI set P_j belonging to category c_j using equation (1):

$$\phi_{ij} = \frac{\log_e(f_{p_i} + 1)}{\sum_{i=1}^{|P_j|} \log_e(f_{p_i} + 1)} \quad p_i \in P_j \tag{1}$$

where ϕ_{ij} denotes the probability of POI p_i generation in c_j. We set ϕ_j to be the probability of all POI generation in P_j. We use e as the Euler number.

The reason for selecting each POI in a round-robin manner as the starting point is that not every POI will necessarily appear as a starting point in the generated data. Therefore, the generated data would lack the data of certain POIs being used as starting points. Following the rules mentioned above, we generate trajectories, denoted as $vtra$ with various lengths ranging from three to the maximum length of trajectories in the dataset. For each length, we generate ρ augmented trajectories. This process ensures that we have a diverse set of augmented trajectories with different lengths, incorporating a wide range of experiences from the original dataset.

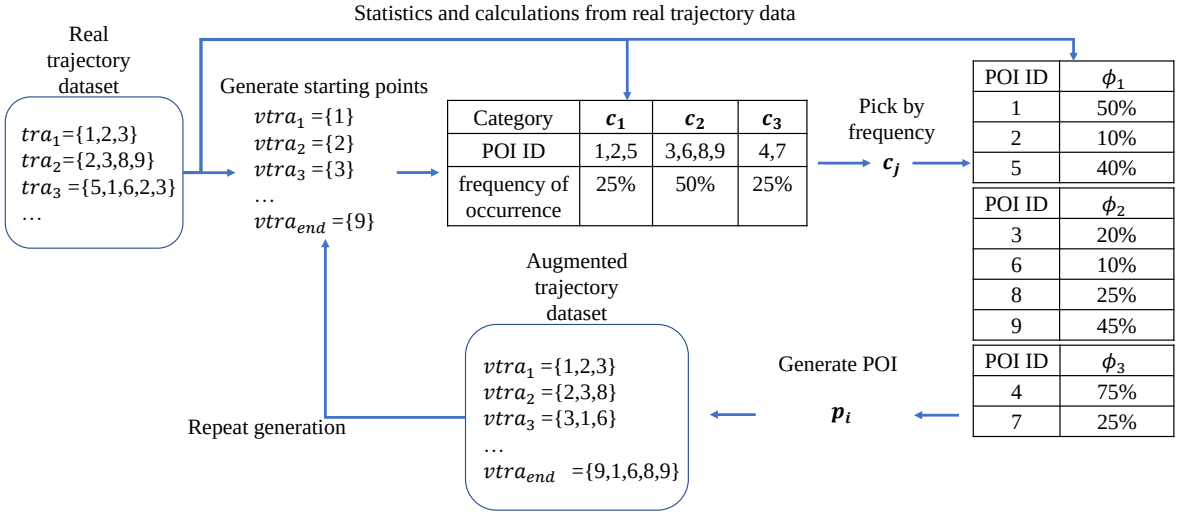


Figure 4-2 The example of data augmentation

As shown in Figure 4-2, for example, let's assume that the original trajectory data contains a total of 3 categories and 9 different POIs. We will calculate the frequency of occurrence for these 3 categories in the dataset and use it as the basis for generating POIs. First, we generate starting points based on all POIs. We select a

category c_j based on its occurrence frequency. Then, we calculate the probability ϕ_j of generating a POI belonging to category c_j based on the occurrence frequency of all the POIs within that category. In this way, we can generate p_i based on ϕ_j and repeat these steps to generate different POIs from different categories until we reach the desired trajectory length. When the number of generated trajectories reaches ρ, we increase the generated trajectory length l by 1 until it equals the length of the longest trajectory in the dataset. Therefore, upon completion of data augmentation, we obtain an augmented dataset that has a similar category distribution to the original trajectory data. In our approach, we assume that the distribution of original trajectory categories in the dataset aligns with users' category preferences. Therefore, the augmented dataset, which adheres to the original data's category distribution, also possesses users' preferred characteristics. Additionally, the distribution of POIs within each category is relatively balanced, and the various trajectory lengths are evenly represented. This helps us to use a balanced dataset during subsequent trajectory feature extraction and recommendation generation model training, leading to better results.

4.3 Trajectory Feature Extraction Model

In order to train the recommendation generation model with appropriate data types, we propose an LSTM-based Autoencoder to address this situation. LSTM is a type of RNN commonly used for learning sequence tasks. It effectively captures and retains long-term temporal dependencies while avoiding the vanishing gradient problem. We employ the Autoencoder to transform the trajectory information into low-dimensional vectors, making these vectors meaningful and capable of being reconstructed back to the original states. Specifically, we use an LSTM-based

Autoencoder to model and encode the original trajectory data into low-dimensional vectors, generating a data format that is more conducive for the recommendation generation model to learn from. The Autoencoder also possesses the reconstruction capability, enabling the encoded vectors to retain the properties of the original trajectories. First, we introduce the inputs of the model, followed by describing the architecture of the model.

4.3.1 Model Input

Our model has three types of inputs: (1) POI Trajectory, (2) Category Trajectory, and (3) Starting Point Query.

(1) POI Trajectory: It consists of all POIs $p \in P$ in the dataset, representing the sequence of POIs in a trajectory. Each POI is represented by its ID, denoted as $p.id$. Since the trajectories in the dataset can have different lengths, we pad the trajectory data with zeros to match the length of the longest trajectory. For example, if a trajectory $tra = \{p_1, p_2, p_3\}$ and the longest trajectory length in the dataset is 6, the padded trajectory would be $\{p_1, p_2, p_3, 0,0,0\}$. After processing all the trajectories, we convert the IDs from categorical format to numerical values by performing one-hot encoding.

(2) Category Trajectory: The category trajectory consists of categories $c \in C$, representing the sequence of categories in a trajectory. It provides information about the category composition, order, and length of the trajectory. Each category is represented by its ID, denoted as $c.id$. Similar to the POI trajectory, we pad the category trajectory to match the length of the longest trajectory. Subsequently, we convert the IDs to numerical values using one-hot encoding.

(3) Starting Point Query: The starting point query refers to the ID of the starting POI,

represented as p_S in the query q. It is also denoted by $p_S.id$. In the training of the Decoder in the Autoencoder, we include this input to enable the model to learn to reconstruct trajectories based on the provided starting point.

These inputs contribute to the training of the model, enabling it to learn and reconstruct trajectories effectively while fusing and preserving essential information.

4.3.2 Model Structure

After introducing the various input data mentioned above, we can input them into the model for training. The goal is to train the model to transform trajectories into low-dimensional features and then reconstruct them back to the original trajectory format. Afterward, we can utilize these low-dimensional features as training data for the next stage. The Autoencoder consists of two main components: the Encoder and the Decoder. In the Encoder, we establish two LSTM layers with different dimensions to extract the inputs along different dimensions. The architecture of our Encoder is shown in Figure 4-3.

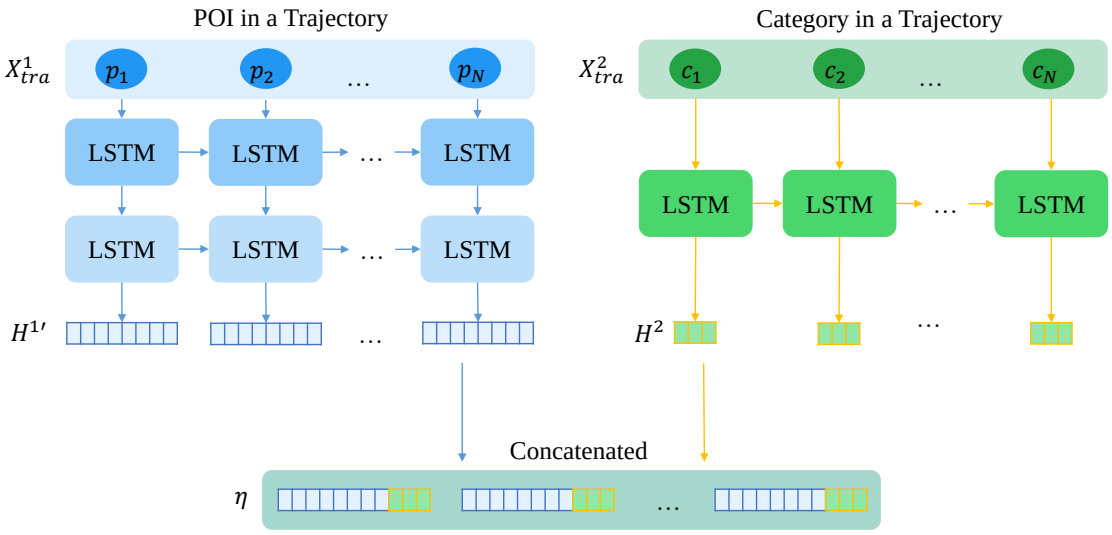


Figure 4-3 The architecture of Encoder

In Chapter 4.3.1, we mentioned the POI trajectory and category trajectory, both of

which consist of a sequence of POIs or categories that form a trajectory. However, these two inputs have significant dimensional differences. Therefore, we employ LSTM units with different dimensions for these two inputs to better capture their respective features along the trajectory.

In the input phase, X_t^1 represents a unit of input, which corresponds to the representation of p in the POI trajectory. However, the POI trajectory consists of multiple POIs, so by combining the representations of the POIs within the trajectory, we can obtain a new input $X_{tra}^1 = [X_t^1, X_{t+1}^1, \dots, X_{l_{max}}^1]$, where l_{max} denotes the longest length among all trajectories. X_{tra}^1 represents all the POI representations within a complete trajectory, and its overall dimension would be $[l_{max} \times d_{POI}]$, where d_{POI} is the one-hot encoding dimension of all POIs.

The input for the category trajectory follows a similar approach. X_{tra}^2 represents a unit of category input, and by combining the representations of the categories within the trajectory, we can obtain a new input $X_{tra}^2 = [X_t^2, X_{t+1}^2, \dots, X_{l_{max}}^2]$. X_{tra}^2 represents all the category representations within a complete tra, and its overall dimension would be $[l_{max} \times d_{category}]$, where $d_{category}$ is the one-hot encoding dimension of all categories.

Next, we feed the POI trajectory and category trajectory inputs into the model and provide a detailed overview of the overall model flow. LSTM units can accept inputs from multiple timesteps and output hidden states h_t, where t represents different timesteps of the input. Each LSTM cell has three gates to determine whether the input should pass through, whether old information should be forgotten, and how new information should be outputted at the current timestep. The states of these three gates at timestep t are represented by equations (2), (3), and (4):

$$f_t = \sigma\big(W_f \cdot [h_{t-1}, x_t] + b_f\big). \quad (2)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i). \quad (3)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o). \quad (4)$$

where f_t, i_t and o_t denote the forget gate, input gate and output gate respectively, σ is the sigmoid function, W and b are weights to be learned and bias, h_{t-1} is the previous hidden state and x_t is the input.

Once the states of these three gates are obtained, we can calculate the output of each cell using equation (5), (6), and (7):

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C). \quad (5)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t. \quad (6)$$

$$h_t = o_t * \tanh(C_t). \quad (7)$$

where W and b are weights to be learned and bias, $*$ means the element-wise multiplication.

Taking X_{tra}^1 as an example, it contains l_{max} units of X_t^1, representing l_{max} LSTM cells. Each cell processes a single timestep input and outputs h_t^1. Finally, the output h_t^1 at each timestep is retained and combined to form a new feature H^1. For the POI trajectory, due to its higher dimensionality, it undergoes two layers of LSTM extraction. The output H^1 from the first layer is then fed as input to the second layer, which outputs $H^{1\prime}$. On the other hand, the category trajectory undergoes a single layer of LSTM extraction, resulting in the output H^2. These outputs, $H^{1\prime}$ and H^2, are concatenated to form an encoded vector η, which serves as the input to the Decoder.

Next, we will introduce our Decoder, Figure 4-4 is our Decoder architecture.

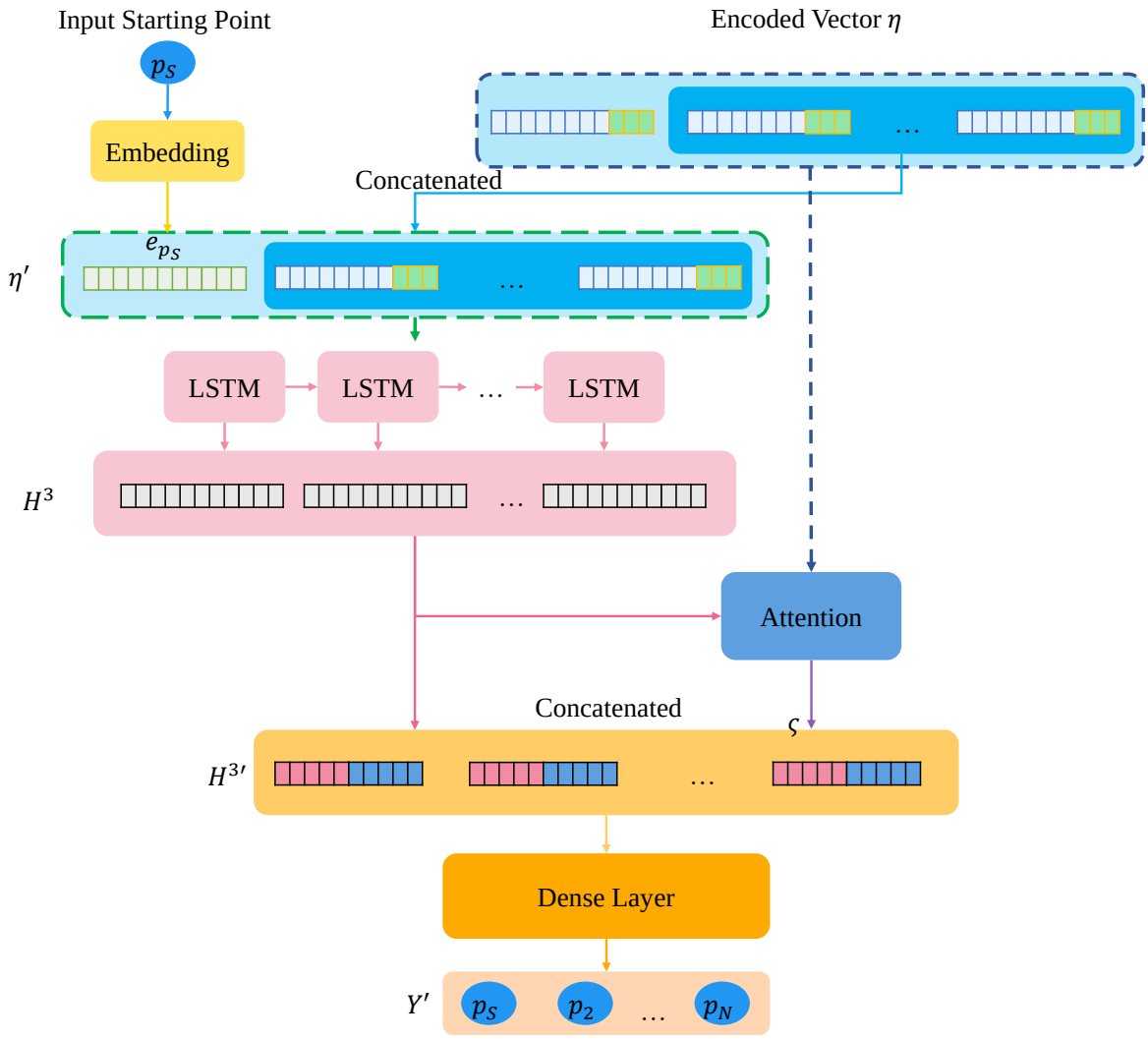


Figure 4-4 The architecture of Decoder

In the Decoder part, we have two inputs: one is the p_S mentioned in Chapter 4.3.1, and the other is the encoded vector η generated from the Encoder. For the p_S input, it is provided as $p_S.id$ without undergoing one-hot encoding like the POI or category trajectories. Once the p_S is inputted, it is embedded into a vector e_{p_S} with the same dimensionality as the encoded vector to enable subsequent concatenation operations. As for the η input, we first exclude the part corresponding to the starting point and replace it with the same dimensionality as e_{p_S}, resulting in a new encoded vector η'. The new encoded vector η' has the same dimension as η.

Next, we input η' into the LSTM-based Decoder, gradually increasing the dimensionality to match the original trajectory input size. The output is denoted as H^3, and similar to the Encoder, we retain the hidden state at each timestep. Additionally, we incorporate an Attention mechanism, which focuses on the most relevant parts and captures contextual information to assist in better reconstructing the trajectory from η and ensuring its coherency. In our model, we consider η as the value and H^3 as the query input to the Attention mechanism. After that, we obtain the context vector ς from the Attention layer, and it is concatenated with H^3 to form $H^{3\prime}$. Finally, $H^{3\prime}$ is directly fed into a Dense layer to restore the dimensionality to that of the original trajectory input, resulting in the output Y' that resembles X_{tra}^1. By applying the softmax activation function, the final output Y' represents the reconstructed trajectory states in the form of POI probabilities. After computing equation (8), we can convert it into a trajectory tra' in the form of $p.id$.

$$tra' = argmax(Y') \tag{8}$$

By utilizing the mechanism of compression and reconstruction through the Autoencoder, we can train the Encoder and Decoder simultaneously and then use them separately after training. The trained trajectory feature extraction model will have the capability to transform trajectory data into encoded vector and reconstruct it back. Our objective is to obtain the encoded vector η. We believe that the extracted trajectory data η contains rich original trajectory features, which can be utilized as a key factor in generating trip recommendations. By using the method described in Chapter 4.2 to augment the training data and then training the trajectory feature extraction model with this augmented data, we can further obtain η, extracted by the Encoder, as the training data for the next stage.

4.4 Recommendation Generation Model

In the recommendation generation model, we chose Conditional Generative Adversarial Network (CGAN) as the main method, which can generate different outputs based on the input conditions. CGAN is an extension of the GAN model and is also composed of a generator and a discriminator. The generator generates outputs based on a random vector r and conditional inputs, and the discriminator distinguishes between the generated outputs and real data. During the training process, the generator and discriminator continuously compete against each other. The generator gradually learns to generate realistic outputs, while the discriminator learns to more accurately differentiate between real and generated data. In this Chapter, we first explain how we design the architecture of CGAN. Then, we provide a detailed overview of the training process. Finally, we explain how we apply our proposed method to perform trip recommendation.

4.4.1 Model Structure

Our proposed CGAN architecture is similar to the previously mentioned Autoencoder, as it also consists of LSTM units and is composed of a generator and a discriminator. The generator takes three inputs: a random vector r, a length condition l and a trip group type g. Figure 4-5 is our generator architecture:

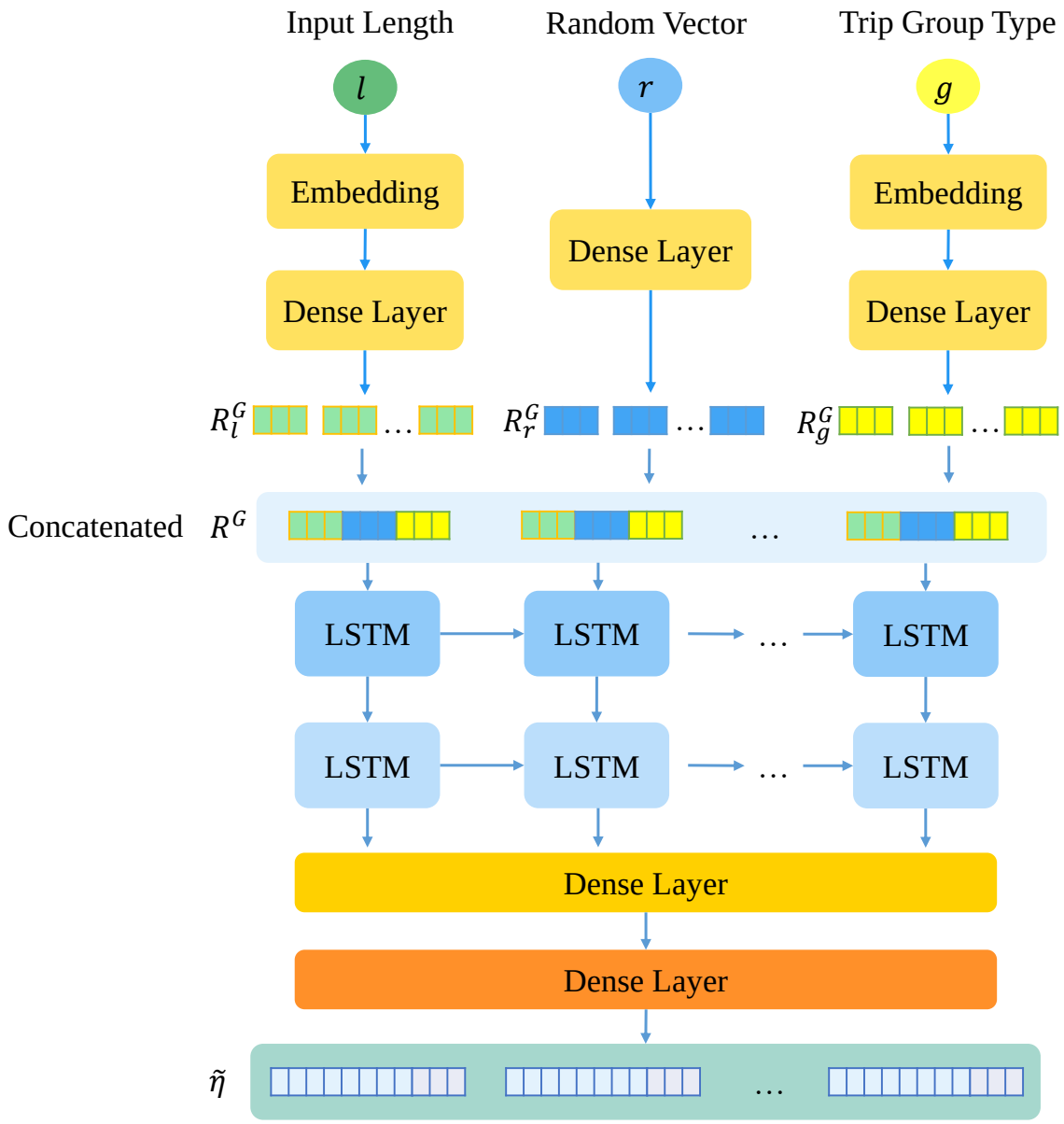


Figure 4-5 The architecture of generator

When inputting into the generator, the random vector r can have any size and is directly fed into a Dense layer, reshaping it into R_r^G with a size of $[l_{max}, d_{\text{random}}]$. Here, $l_{\max}$ represents the longest length of all trajectories, which is also the timesteps of the Encoder output, and d_{random} can be set to any dimension. The other two conditions, l and g, follow a similar process. l denotes the recommended trip length, and g represents different trip group types of trip conditions. Both l and g

are first embedded and then input into a Dense layer, resulting in R_l^G and R_g^G, respectively. Their dimensions are $[l_{\max}, d_l]$ and $[l_{\max}, d_g]$, where d_l and d_g can be set to any dimension. Next, R_r^G, R_l^G and R_g^G are concatenated to form R^G, which is then input into an LSTM layer. This allows the model to learn the transformation from the random vector and the condition inputs to the encoded representation of trajectories. Finally, after passing through a Dense layer, a generated encoded vector $\tilde{\eta}$ similar to η is produced. In the next stage, we will introduce the discriminator. Figure 4-6 presents the architecture diagram of the discriminator.

The main purpose of the discriminator is to distinguish between real and fake inputs and check if the conditions match. Similar to the generator, the discriminator also has three inputs: the target to be discriminated t, the length condition l, and the trip group types g. First, the input t can be either the real data η from the Encoder or the fake data $\tilde{\eta}$ from the Generator. Therefore, its dimension is the same as $\tilde{\eta}$. And the inputs l and g undergo the same process as in the Generator. Both of them are first embedded and then fed into a Dense layer, followed by reshaping into R_l^D and R_g^D, respectively. The dimensions of R_l^D and R_g^D are $[l_{\max}, d_l]$ and $[l_{\max}, d_g]$, where d_l and d_g can be set to any dimension. Afterward, t, R_l^D and R_g^D are concatenated into R^D and input into two layers of LSTM to learn to discriminate between real and fake data. Finally, all the information is flattened and fed into a Dense layer to output the discrimination result.

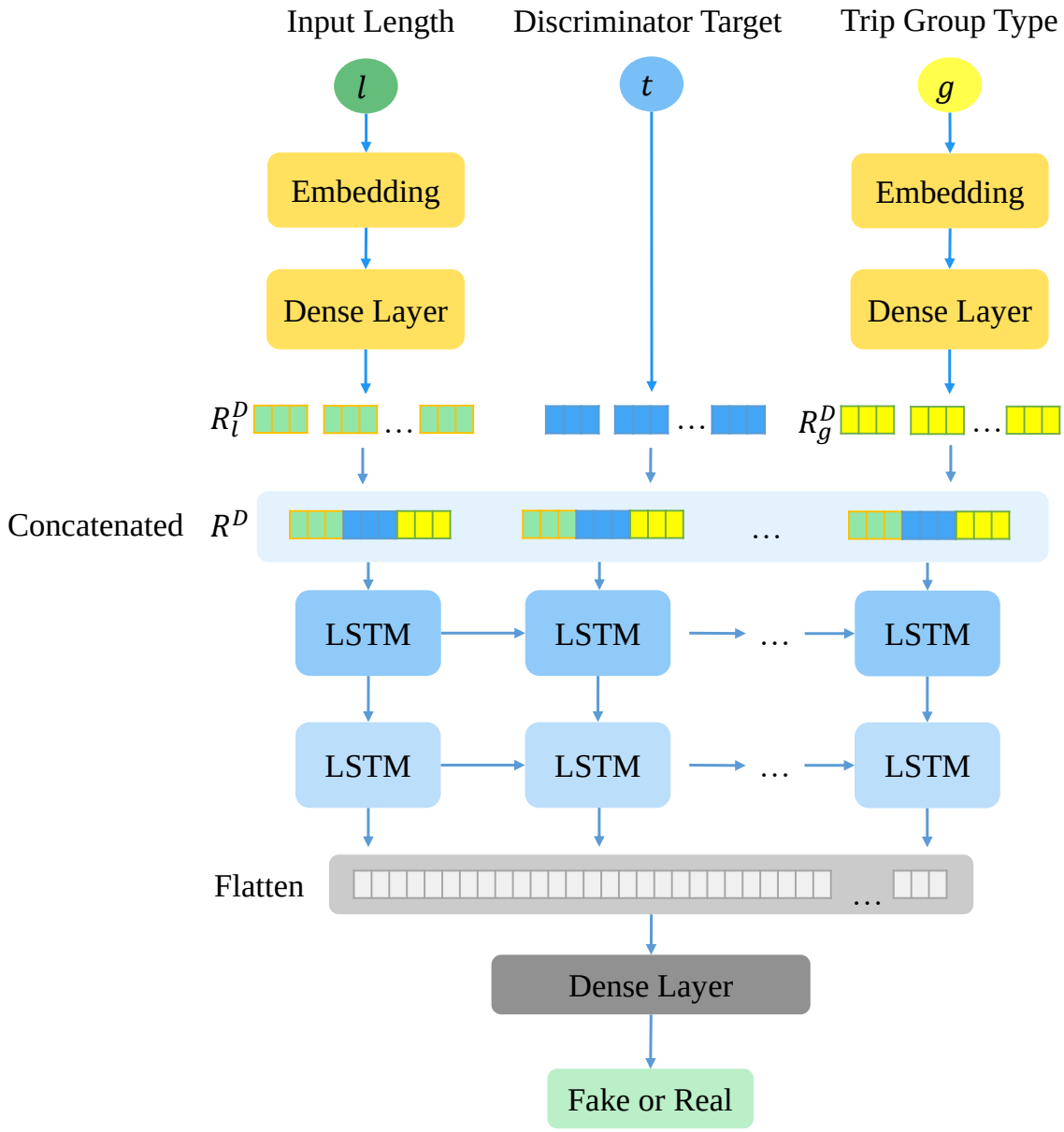


Figure 4-6 The architecture of discriminator

4.4.2 Training Process

After introducing the model architecture, we will now explain the setup of input conditions and the training process. In the biginning, we use the trained trajectory feature extraction model to generate the real data, which is the encoded vector η required for CGAN. Then, based on these vectors η and their original trajectory lengths, we create the input data for the length condition, denoted as l. Moreover, we

create a negative sample length input, denoted as l_N, based on these lengths. Specifically, l_N is also a length input, but it represents incorrect lengths different from the correct ones, randomly selected. The purpose of the negative sample is to serve as fake data input to the discriminator, allowing it to recognize that other lengths are incorrect and should be classified as fake, thereby reinforcing the accuracy of the length input in the generator. For example, when the encoded vector with a length of Z is input, the negative sample length input would be as equation (9).

$$If\ l = Z, l_N = M, Z \notin M \tag{9}$$

In terms of the trip group type condition g, we have defined H types of inputs: Type A, Type B,..., and All Types. These types are derived from clustering the users based on the proportions of different POI categories they have visited in the original trajectory data. Specifically, we first use the proportions of various POI categories visited by each user u as input and perform K-means [25] clustering on all users. By analyzing the Silhouette score [28], we determine that $K = H - 1$ is the most appropriate number of clusters. Consequently, all users are divided into H groups, and the trajectory data is also divided into H groups. However, we also consider that not all users may want to choose specific trip group types of trip conditions. Therefore, we introduce the All Types condition, which includes categories from all $H - 1$ types, allowing users to have a more flexible choice.

Once these conditions are established, we can begin training the proposed CGAN model. Referring to [9], a GAN consists of a generator $G(\cdot)$ and a discriminator $D(\cdot)$, which engage in a two-player minimax game based on equation (10).

$$\begin{aligned}\min_G \max_D V(D,G) \\ = \mathbb{E}_{x\sim p_{data}(x)}[log\, D(x)] + \mathbb{E}_{z\sim p_z(z)}[log(1 - D(G(z)))].\end{aligned} \tag{10}$$

Based on this value function, incorporating the conditional inputs, we can rewrite it

as equation (11).

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[log\, D(x|y)] + \mathbb{E}_{z\sim p_z(z)}[log(1-D(G(z|y)))]. \quad (11)$$

Based on the aforementioned equation, we attempt to apply our proposed inputs to the CGAN. Specifically, we define the encoded real trajectory data, the encoded vector η, as the ground truth, denoted as $x^{(i)}$. For the generator, we input the length condition and trip group type condition, denoted as $y^{(i)}$. For the discriminator, we input the generated encoded vector $G(z^{(i)}|y^{(i)})$ from the generator, the complete set of conditional inputs $y^{(i)}$, the negative sample conditional input $ny^{(i)}$, and the real encoded vector $x^{(i)}$. Therefore, our loss function simultaneously trains the generator and discriminator and can be rewritten as equation (12) and (13).

$$L_G = \frac{1}{m}\sum_{i=1}^{m} log\left(1 - D\left(G\left(z^{(i)}\middle|y^{(i)}\right)\middle|y^{(i)}\right)\right). \quad (12)$$

$$L_D = \frac{1}{m}\sum_{i=1}^{m}[-log\, D\left(x^{(i)}\middle|y^{(i)}\right) - log\left(1 - D\left(x^{(i)}\middle|ny^{(i)}\right)\right) - log\left(1 - D\left(G\left(z^{(i)}\middle|y^{(i)}\right)\middle|y^{(i)}\right)\right)]. \quad (13)$$

where m is the mini batch size. We use binary cross entropy to minimize our loss function and adopt Adam optimizer to train our model.

4.4.3 Recommendation Process

After data augmenting, trajectory feature extraction model and recommendation generation model training, we can obtain a Decoder that can restore the encoded vector η back to the original trajectory tra, as well as a generator that can generate $\tilde{\eta}$ based on specific input conditions. By applying these two models, we can provide

diverse trip recommendations. The detailed process is illustrated in Figure 4-7.

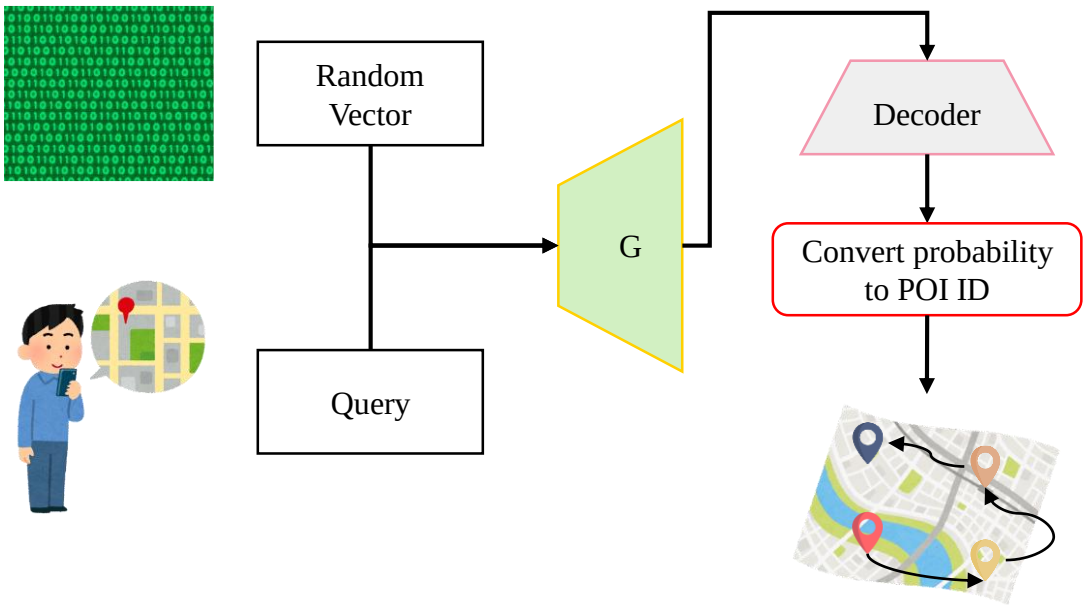


Figure 4-7 The recommendation process

Specifically, we can collect information from the user's end, such as the starting location, the number of desired POIs to visit, and the preferred trip group type, and combine them into a query q. Next, we input the random vector r, the conditions l and g into the generator to generate a encoded vector $\tilde{\eta}$ that meets the specified conditions. Then, we feed $\tilde{\eta}$ and p_s into the Decoder, allowing the Decoder to restore the input back to the trajectory state in the form of POI probabilities. After that, we convert it into a sequence of $p.id$ using equation(8). This sequence serves as the recommended trip trajectory.

Our proposed method can generate diverse trip trajectories by leveraging the random input feature of CGAN, ensuring that the recommended POIs vary each time and providing users with different experiences. This characteristic significantly differs from previous studies that focused on predicting user trajectories, opening up new research directions in the field of recommendation. Additionally, our method can

generate recommendations that align with user expectations to some extent by incorporating input conditions. Similar constraints have been applied in previous literature to flexibly handle the recommendation process. In practical applications, appropriate conditions can effectively assist users in obtaining recommendations that meet their needs.

Chapter 5 Experimental Evaluation

In this chapter, we discuss the experiments and analysis of our proposed method. This chapter is divided into four parts. Chapter 5.1 describes the detailed data sources, preprocessing parameters, evaluation metric calculations, and experimental setup. Chapter 5.2 explains the performance of our method under different parameter settings and analyzes the purposes of the parameters. Chapter 5.3 presents the comparison between our proposed method and other trip recommendation methods. Chapter 5.4 shows the practical results of utilizing deep learning methods and our method across different datasets for recommendations.

5.1 Experimental Data and Setting

The data we used in the experiment mainly comes from the Foursquare Dataset [37], summarized by Gao et al. [14]. It includes long-term (about 10 months) check-in data in Tokyo collected from April 2012 to February 2013. The detailed contents of the dataset are shown in Table 5-1.

Table 5-1 The detailed contents of the Tokyo Dataset

Tokyo Dataset	
City	Tokyo
Duration	April 2012 to February 2013
User	44
Trajectory (>3 POIs)	1,680
POI	3,680
Category	14

Due to the large number of POIs covered in the original check-in dataset and the scarcity of trajectory data, the scale is unbalanced for training, and applying other methods from the literature takes excessive time. Therefore, we only use a subset of

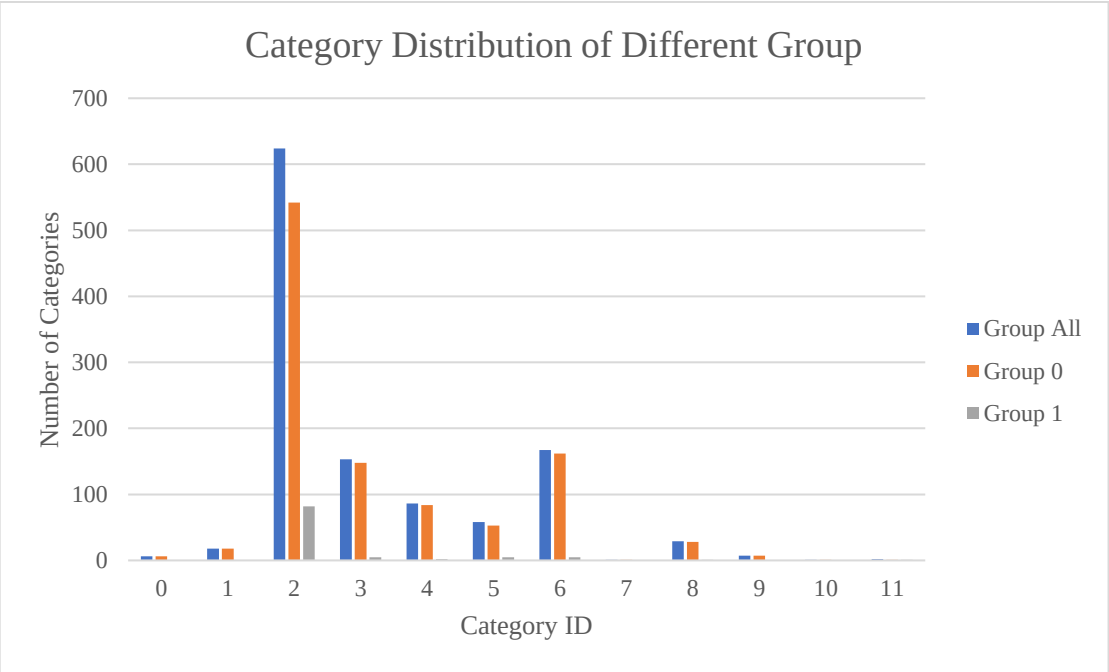
the POIs and check-in trajectory data for our experiments. Specifically, from the 1680 trajectories in the Tokyo dataset, we select the most frequent POI, denoted as p_f, as the base. We then extract a new trajectory dataset consisting of only those trajectories that contain p_f. After preprocessing, the detailed content of the dataset is shown in Table 5-2. We refer to this dataset as the Tokyo Small Dataset.

Table 5-2 The detailed contents of the Tokyo Small Dataset

Tokyo Small Dataset	
City	Tokyo
Duration	April 2012 to February 2013
User	25
Trajectory (>3 POIs)	214
POI	367
Category	12

The Tokyo Small Dataset consists of 367 POIs, 25 users, and 214 trajectory data. The shortest trajectory length is 3, and the longest trajectory length is 19. The dataset includes 12 different POI categories. After obtaining the Tokyo Small Dataset, we apply the data augmentation method mentioned in Chapter 4.2 to expand our dataset. First, we use the distribution of categories among users as the basis for selecting the trip group type g. We calculate the frequency of each category visited by each user and convert these frequencies into probability form. We then perform K-means [25] clustering on these probabilities. Based on the Silhouette score [28] calculation, we find that dividing the users into 2 groups is the most suitable. Additionally, we add an additional option for users who don't want to choose a specific trip group type, which is a non-clustered option. This option is created by combining the category distributions of the two groups. The category distributions of each group are shown in Table 5-3.

Table 5-3 The category distributions of different group



Next, we perform data augmentation on the three groups separately. We set $\rho = 500$, meaning that for each trajectory length, we generate 500 augmented trajectories for each group, resulting in a total of 25,500 augmented trajectories. These augmented data are used solely for training the models. Once the training converges, we conduct internal experiments using various strategies and external experiments by comparing our approach with different trip recommendation methods.

In the experimental setup, all experiments were conducted on a system with an Intel Core i9-10900K CPU clocked at 3.70GHz, 64GB RAM, and NVIDIA GeForce RTX 3090 running on Microsoft Windows 10. The expanded dataset of 25,500 trajectory sequences was divided into training and testing data. We allocated 80% of the trajectory sequences for training and the remaining 20% for testing. We implemented and tested our proposed model using Python 3.9 and Keras 2.6.0.

As we stated in Chapter 4.2, we assume that the category distribution contained in the dataset we use is user-preferred, so we will use the original POI category

distribution in the dataset as the basis for the metrics. These experiments utilize several metrics that can be categorized into four dimensions: diversity, accuracy, redundancy, and stability. In the following, we will provide explanations for these four metrics.

I. Diversity

In terms of diversity, we choose to use entropy to measure the overall randomness of the recommended trips. The calculation is performed using equation (14):

$$\varepsilon = -\sum_{i=1}^{|P|} \frac{f_{p_i}}{\sum_{k=1}^{|P_j|} f_{p_k}} \log \frac{f_{p_i}}{\sum_{k=1}^{|P_j|} f_{p_k}} \tag{14}$$

where $|P|$ is the number of all POIs appearing in the recommended trip, $|P_j|$ is the number of all POIs in the category c_j and $p_i \in P_j$. In this metric, the object of our calculation is the probability of each POI participating in the overall confusion in the recommendation results. The larger the value of ε, the more chaotic and diverse the recommended trajectory will be.

II. Accuracy

In the accuracy part, we use Jensen-Shannon Divergence (JS-divergence) to assess the accuracy between the recommended trajectory and the original data's trajectories. Specifically, we calculate the JS-divergence using equation (15) between the probability distributions of POI categories in the original data and the recommended trajectory.

$$D_{JS}(O \parallel T) = \frac{1}{2}\left(D_{KL}(O \parallel M) + D_{KL}(T \parallel M)\right) \tag{15}$$

where O is the probability distribution of POI categories of users in the original data, T is the probability distribution of POI categories of the recommended

trajectories, $M = \frac{1}{2}(O + T)$ is a mixed distribution and D_{KL} is KL-divergence. In this metric, the object of our calculation is the probability distribution of POI categories appearing in all recommendation results. The degree of similarity between the category probability distribution of recommended trips and the category probability distribution of the original data can be obtained through calculation. A lower value of D_{JS} indicates a smaller difference between the two distributions, indicating similar characteristics. This implies that the recommended trips align better with the user preferences.

III. Redundancy

When we mention redundancy, we define the Repeat Ratio RR as the proportion of trajectories in the recommended trips that have duplicate POIs. Specifically, we set the number of trajectories with repeated POIs as N_R, and the total number of test trajectories as N_T, and the calculation of RR is shown in equation (16).

$$RR = \frac{N_R}{N_T} \tag{16}$$

IV. Stability

Stability is mainly to test whether our recommendation method can recommend a suitable trajectory according to the conditions. Specifically, we evaluate this through the use of boxplots for trajectory lengths, and Starting Point Accuracy SA. The boxplots provide an overview of the distribution of trip lengths generated by our method, while the SA indicates how accurately the recommended starting points match the given conditions.

In the autoencoder component, we employed categorical cross-entropy as the loss function for minimizing reconstruction error. The Adam optimizer was used

during training, and a total of 70 epochs were trained. For the Encoder component, we set different extraction dimensions based on the input configurations. Specifically, the LSTM for extracting POI trajectories was set to 128 and 64 dimensions, while the LSTM for extracting category trajectories was set to 5 dimensions. Therefore, the concatenated encoded vector had a total dimension of 69. In the Decoder component, as the embedded features were concatenated with the encoded vector, we set the dimension to 69. The subsequent LSTM was also set to 69 dimensions. Finally, for reconstructing the trajectories, we set the dimension of the Dense layer to match the total number of POIs, which was 368. We applied a softmax activation function to convert the reconstructed trajectories into probability distributions over each POI. Detailed dimension settings are illustrated in Figure 5-1 and Figure 5-2.

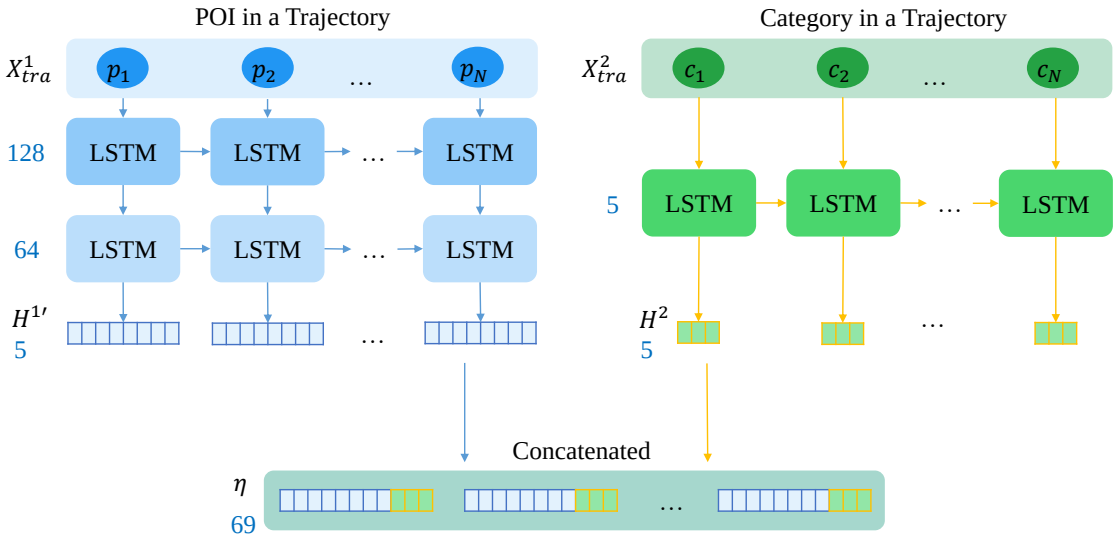


Figure 5-1 The dimension settings of Encoder

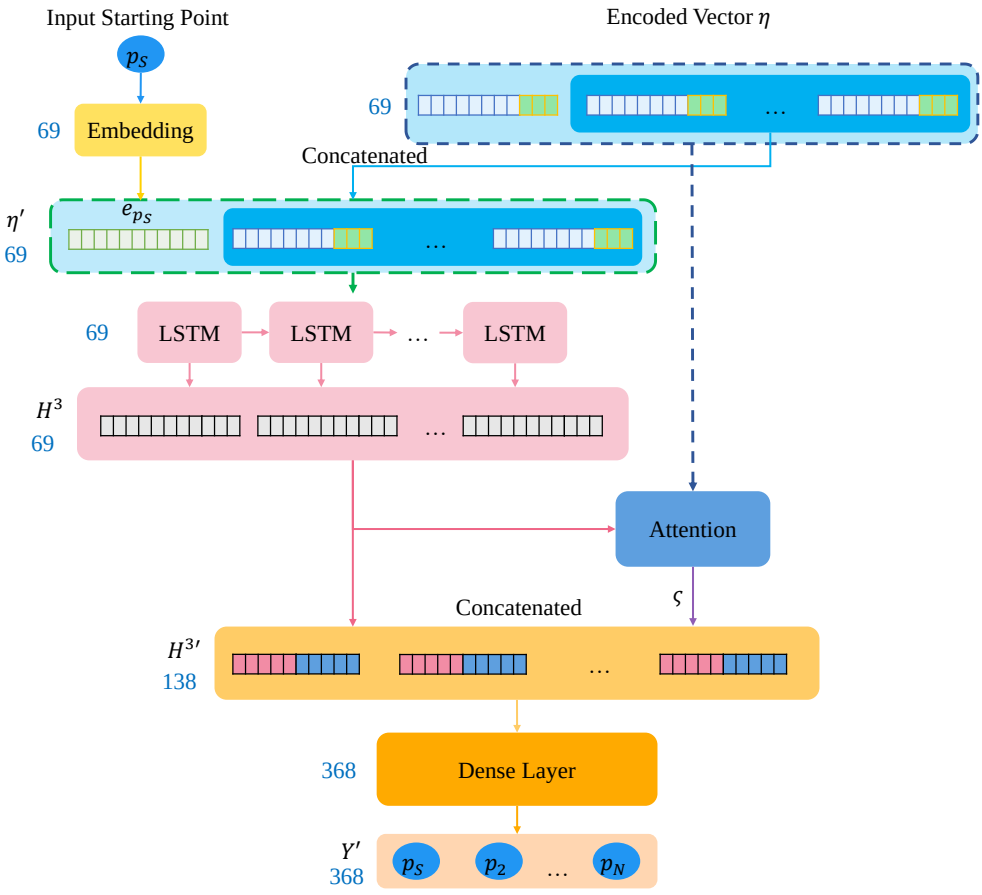


Figure 5-2 The dimension settings of Decoder

In the CGAN component, we employed binary cross-entropy as the loss function to minimize. Similar to the autoencoder, we used the Adam optimizer and trained the model for 50 epochs. The dimension of the input random vector was set to 100. For the length condition input, we used an embedding size of 10, and for the trip group type condition input, we used an embedding size of 5. In the generator, we set the number of LSTM units to 256 and 128, and the connected Dense layer had dimensions of 128 and 69, matching the dimension of the encoded vector η. In the discriminator, we set the number of LSTM units to 128 for both layers. The final Dense layer was configured with a single unit for binary classification (real or fake) and employed the sigmoid activation function. Detailed dimension settings can be found in Figure 5-3.

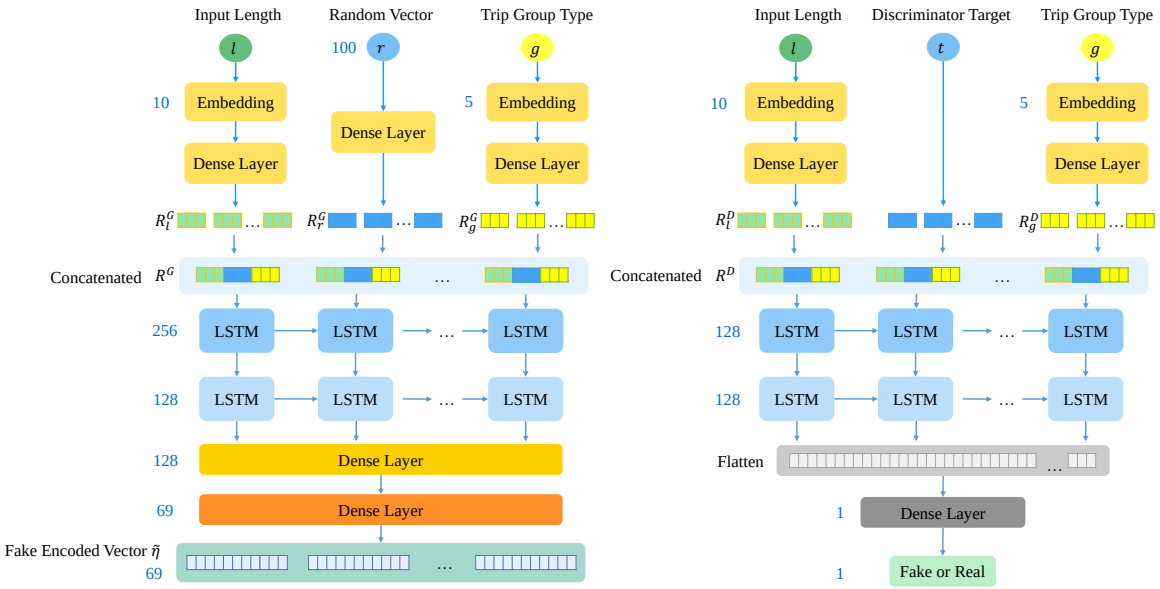


Figure 5-3 The dimension settings of CGAN

5.2 Internal Experiment

In the internal experiments, we conducted evaluations based on six different parameter settings: data augmentation mechanism, data augmentation limitations, negative samples, recommendation selection strategy, and model structure. The details of these settings and the corresponding parameters used are provided in Table 5-4. All internal experiments were conducted using the same input values for p_s, l, g and r. We set g to Group All and generated recommendations for each trajectory length l 100 times. This means we performed experiments from length 3 to 19, resulting in a total of 1,700 experiment trials. We evaluated the performance of various metrics within these parameter settings and analyzed the results of these experiments.

Table 5-4 The settings of the internal experiment.

Parameter	Value	Default
Data Augmentation Mechanism	Hot Random HotLN	HotLN
Data Augmentation Limitations	Allowing POI Duplication Not Allowing POI Duplication	Not Allowing
Negative Samples	Yes No	Yes
Recommendation Selection Strategy	Select All Non-empty Value Select Until The First Empty Value	Select All Non-empty Value
Model Structure	Has Attention Remove Attention	Has Attention
	LSTM GRU	LSTM
	Code size:69 Code size:133	Code size:69

5.2.1 Data Augmentation Mechanism

In this experiment, we tested the training performance of the model under different data augmentation mechanisms to compare their effectiveness. We evaluated three mechanisms: Hot, Random, and HotLN. Hot means that we directly use the frequency of each POI in the selected category c_j as the selection probability during data augmentation. Random denotes that we randomly select a POI from the selected category c_j during data augmentation. HotLN represents that we use the probability calculated by equation (1) proposed in Chapter 4.2 for selecting POIs during data augmentation. The experimental results are shown in Figure 5-4, which includes the values of ε、D_{JS}、RR and SA for each mechanism. In Figure 5-5, we plotted boxplots for each mechanism when generating trajectories of different lengths.

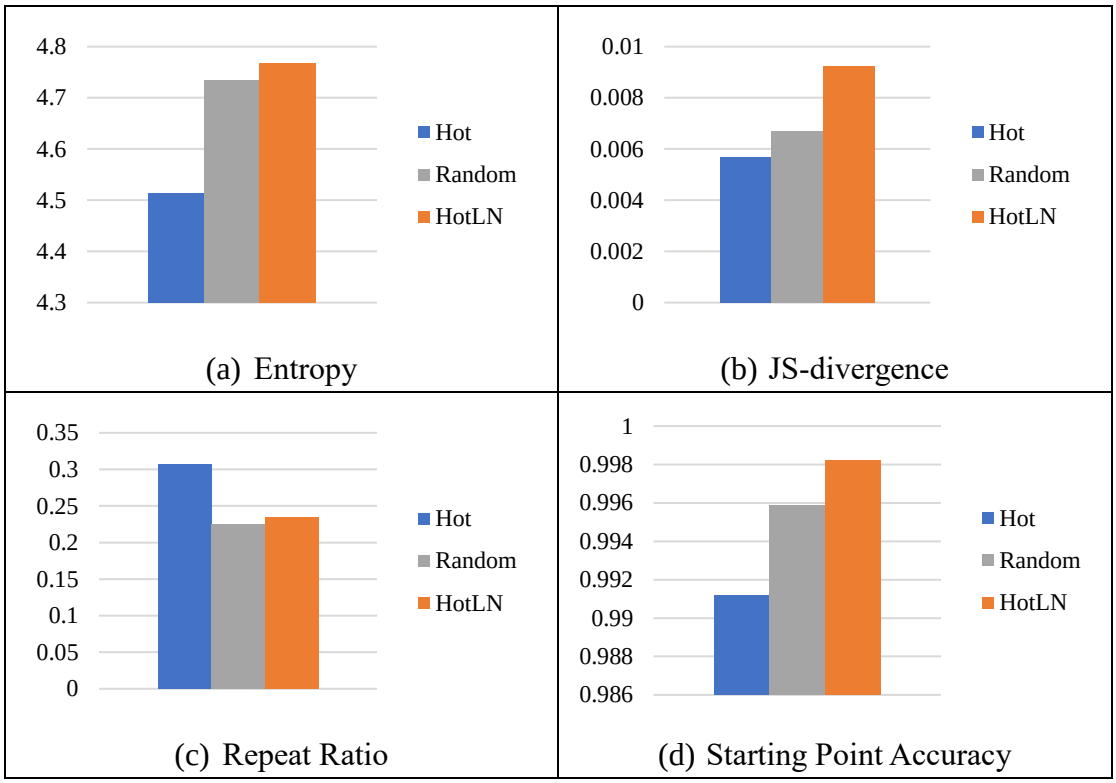


Figure 5-4 The results of different data augmentation mechanisms

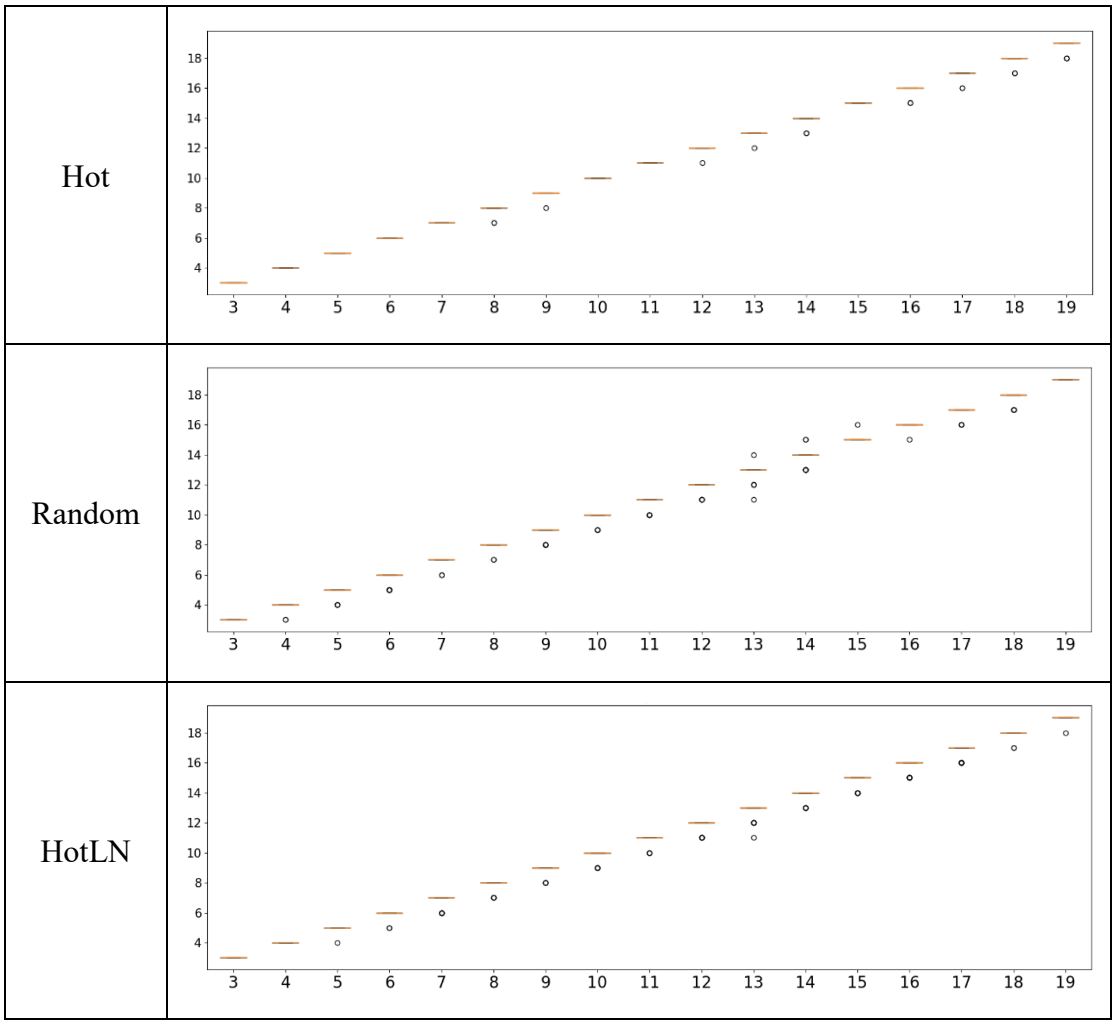


Figure 5-5 The boxplots of different data augmentation mechanisms

In Figure 5-4, we found that HotLN is not as accurate as the other two mechanisms. We believe this is because when generating trajectory data, there is a restriction that prevents the generation of duplicate POIs, which alters the distribution and affects the distribution of categories. However, in terms of diversity, HotLN outperforms the other mechanisms, even surpassing Random, and it also has a lower repeat ratio. We suppose this is because the trajectory augmentation data generated by this mechanism avoids the repetition of the same POI due to disparate probabilities, while maintaining the relative distribution of POIs in the original data. This allows HotLN to strike a balance between diversity and accuracy, without biasing towards generating POIs with abnormally high probabilities. Besides, in Figure 5-5, when it

comes to recommendation generation, HotLN exhibits stability in trajectory length compared to the fluctuating behavior of the Random mechanism. It demonstrates consistent generation characteristics. Based on these observations, we choose HotLN as the primary data augmentation mechanism.

5.2.2 Data Augmentation Limitations

In the second experiment, we aimed to investigate whether restricting the generation of augmented trajectories to avoid duplicate POIs would affect the overall recommendation performance of the model. Trajectory data with duplicate POIs theoretically reduces the diversity of generated trajectories during training. However, due to the inherent randomness in CGAN's inputs, the actual impact of this limitation needs to be evaluated using performance metrics. Figure 5-6 presents the performance of various metrics under these two limitations, and Figure 5-7 visualizes the stability of trajectory lengths under the two limitations.

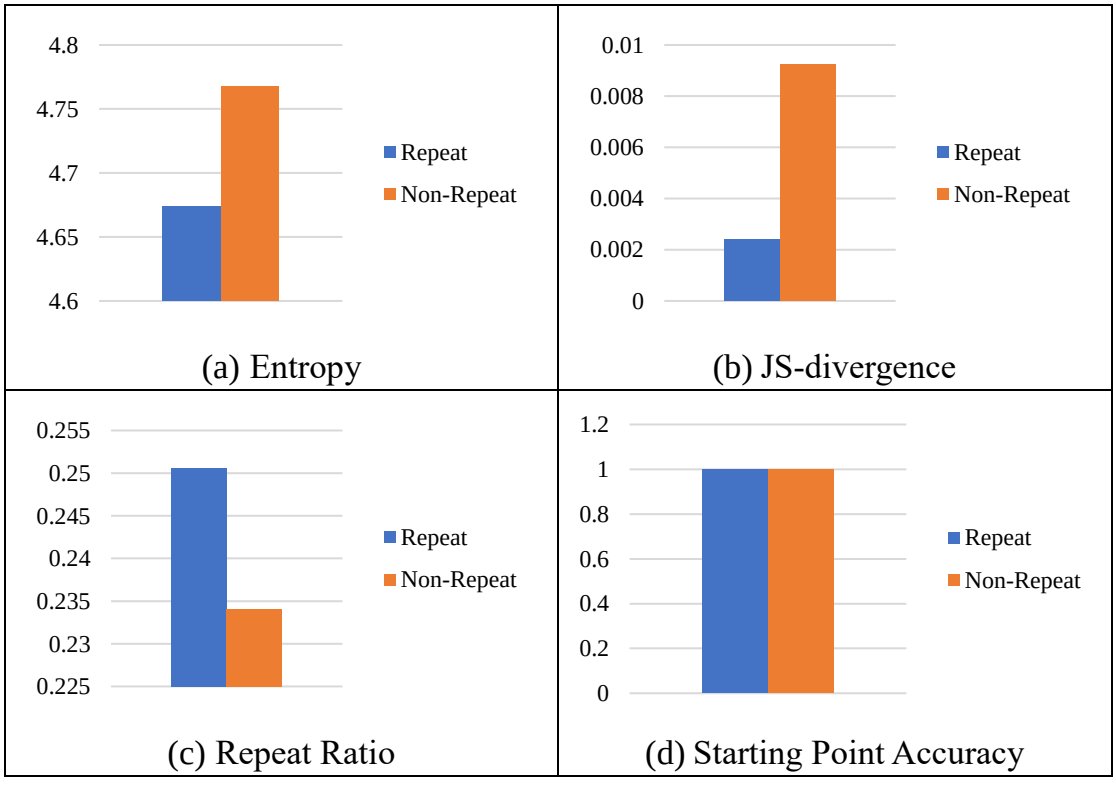


Figure 5-6 The results of different data augmentation limitations

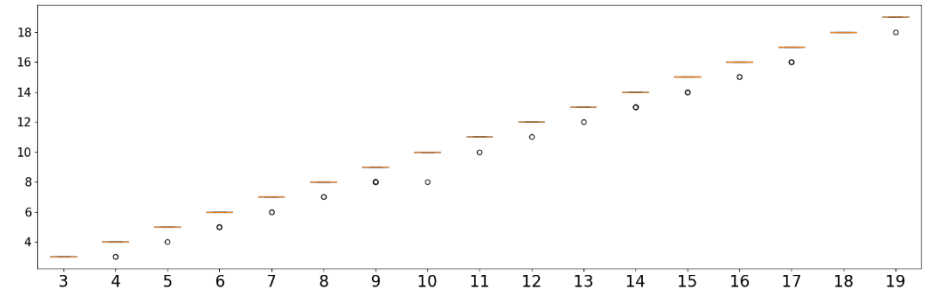
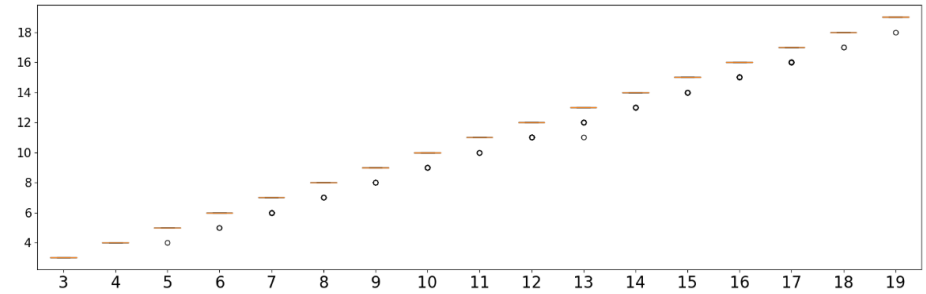
Repeat	
Non-Repeat	

Figure 5-7 The boxplots of different data augmentation limitations

The limitation that allows duplicate POIs results in better accuracy. In terms of diversity metrics, it indeed shows lower values, and the repeat ratio is higher compared to the limitation that prohibits duplicates. We believe this is due to the reasons mentioned in Chapter 5.2.1, where the limitation on duplicate POIs leads to the exclusion of many duplicate trajectory data that still align with the distribution, resulting in slight differences in the category distribution compared to the original data. On the other hand, allowing duplicate POIs enables the generation of augmented trajectory data based on the category distribution, leading to a smaller distribution distance from the original data and higher accuracy. Regarding the stability of trajectory lengths, there is no significant difference between the two limitations. In terms of diversity, the limitation that prohibits duplicates demonstrates better performance. Thus, we choose the limitation on duplicate POIs as our default parameter.

5.2.3 Negative Samples

In the third internal experiment, we compared the difference between the metrics with and without negative samples. Negative samples are primarily used during the training phase of CGAN, where erroneous conditions are paired with real data. These sample can provide the discriminator with more combinations of data and conditions to reference and improve the performance of the generator. In this experiment, we aimed to test whether the inclusion of negative samples in model training could improve the issue of length instability. Figure 5-8 and Figure 5-9 display the results of various metrics and the length stability, respectively.

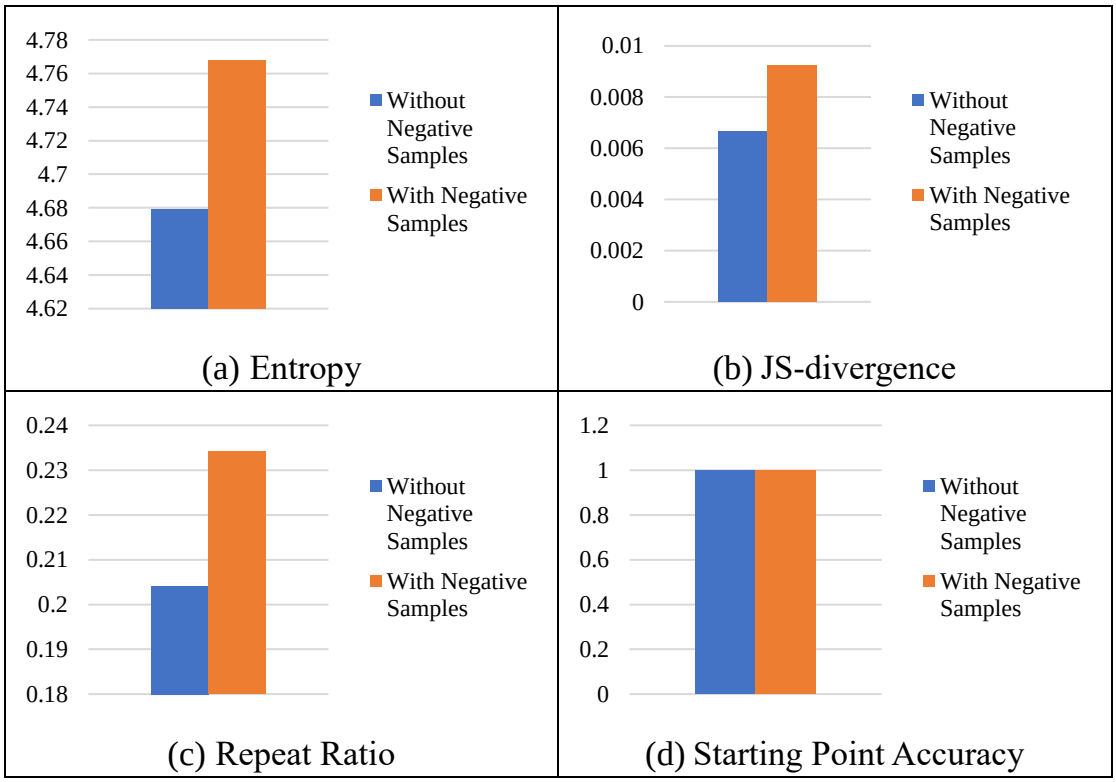


Figure 5-8 The results of model with and without negative samples

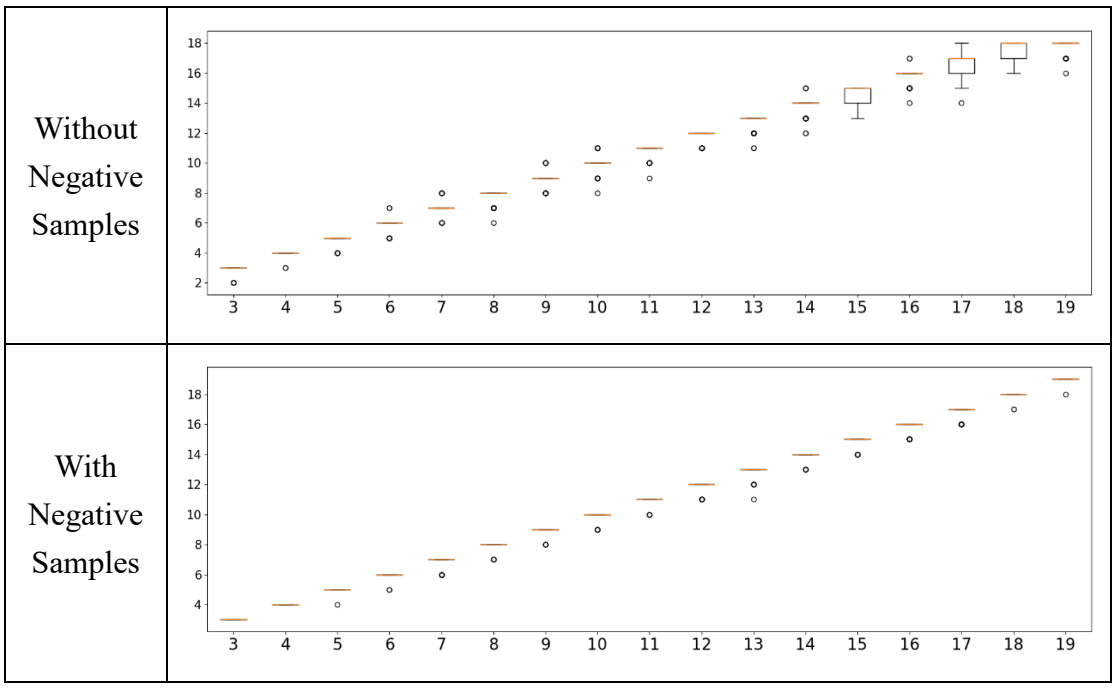


Figure 5-9 The boxplots of model with and without negative samples

For accuracy performance, there is not much difference between the two approaches. However, when it comes to redundancy, the inclusion of negative samples does not significantly improve the redundancy. The performance in terms of the starting point accuracy is comparable between the two approaches, but the model with negative samples exhibits higher diversity. The most significant improvement is observed in length stability. From Figure 5-9, it is evident that the experiment without negative samples shows less stability, with an increase in outliers and occurrences of values greater or smaller than the condition. The performance also fluctuates within a single length, resulting in a larger variation range in the boxplot. Therefore, we can conclude that the inclusion of negative samples effectively improves length stability, making the generated outputs more aligned with the conditions. Hence, we choose to use negative samples as the default setting.

5.2.4 Recommendation Selection Strategy

In the fourth experiment, we wonder about the impact of different value selection strategies in the recommended outputs on various indicators. After the recommendation process by CGAN and Autoencoder, we obtained recommended sequences of a fixed length of 19, where 0 represents an empty value. Therefore, shorter recommended sequences tend to have more empty values. However, in some sequences, 0 may appear in the middle of longer recommended sequences, indicating that we can adopt different value selection strategies for these sequences. The first strategy is to only consider values until the first empty value appears, while the other strategy is to include all non-empty values. For convenience, we called the first strategy as FE and the second strategy as ANE. Figure 5-10 and Figure 5-11 show the performance of various metrics for these two strategies.

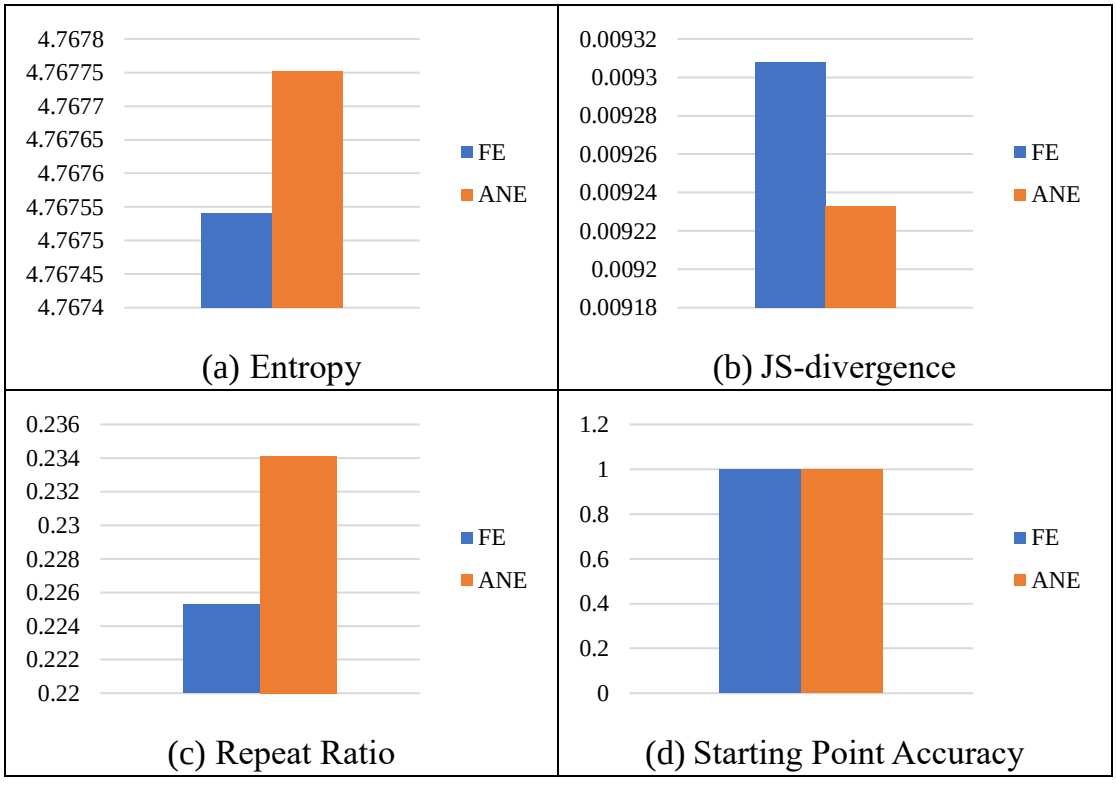


Figure 5-10 The results of different recommendation selection strategies

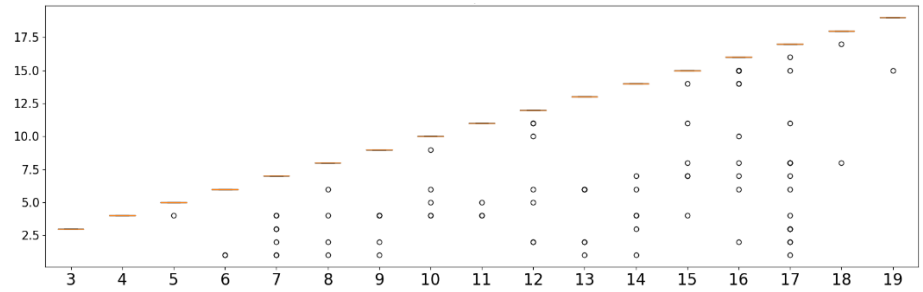
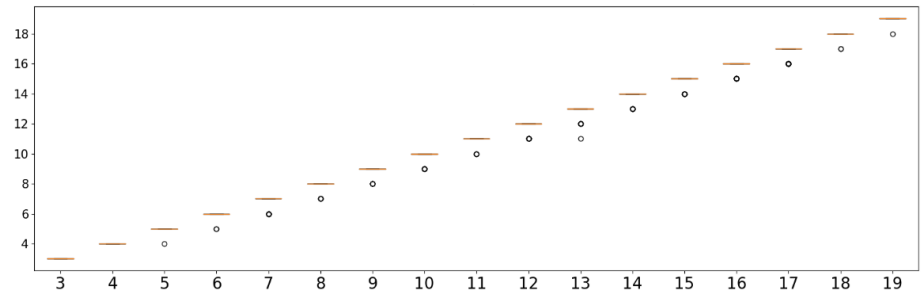
Select Until The First Empty Value	
Select All Non-empty Value	

Figure 5-11 The boxplots of different recommendation selection strategies

There is no difference in the starting point accuracy between the two strategies in Figure 5-10. In terms of accuracy and diversity, ANE is slightly better than FE. For the repeat ratio, FE is significantly lower than ANE. Because of the longer conditions, there is a higher chance of encountering empty values, which leads to shorter recommended trips with reduced opportunities for repetition in single recommendations. From Figure 5-11, we also observe similar results due to the same reason. FE results in various outliers under longer length conditions and exhibits poorer stability. Therefore, we choose ANE as the default.

5.2.5 Model Structure

In the fifth experiment, we tested the impact of different model parameters on the metrics. In the beginning, we examined the performance before and after incorporating the Attention mechanism. Attention allows the model to dynamically focus on relevant information at different input positions based on their importance. With all other parameters fixed, we trained the model by including only the Attention

component. The results are shown in Figure 5-12 and Figure 5-13.

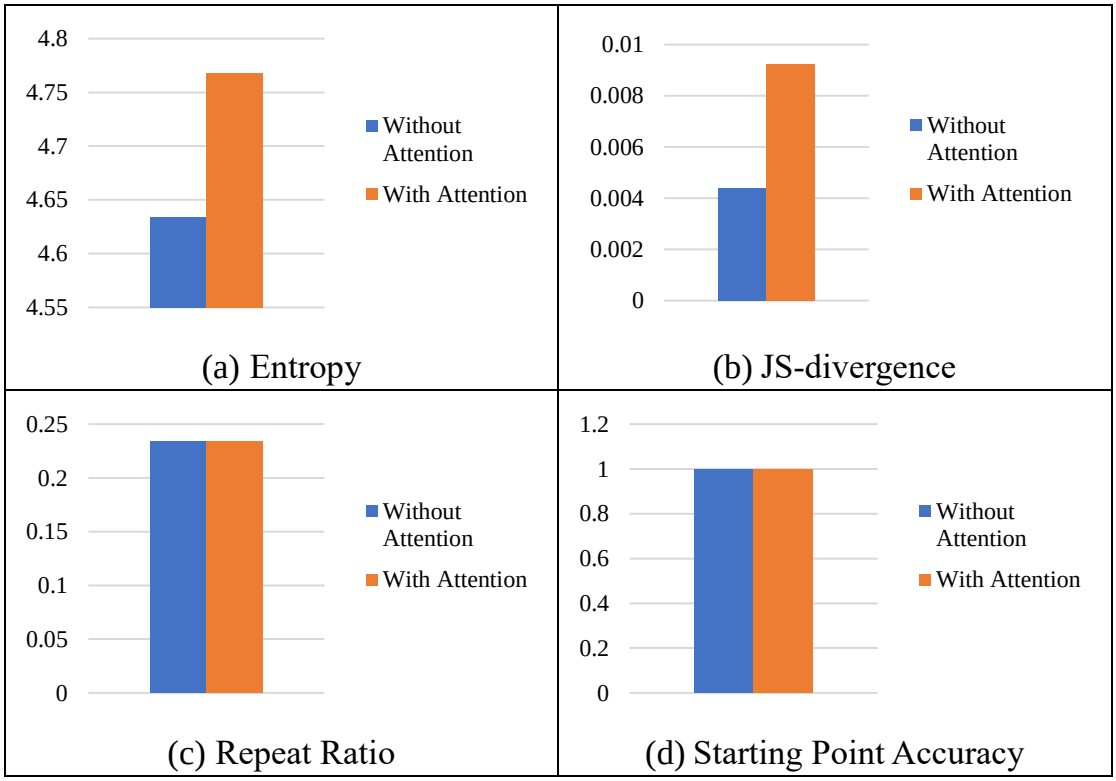


Figure 5-12 The results of model with and without Attention

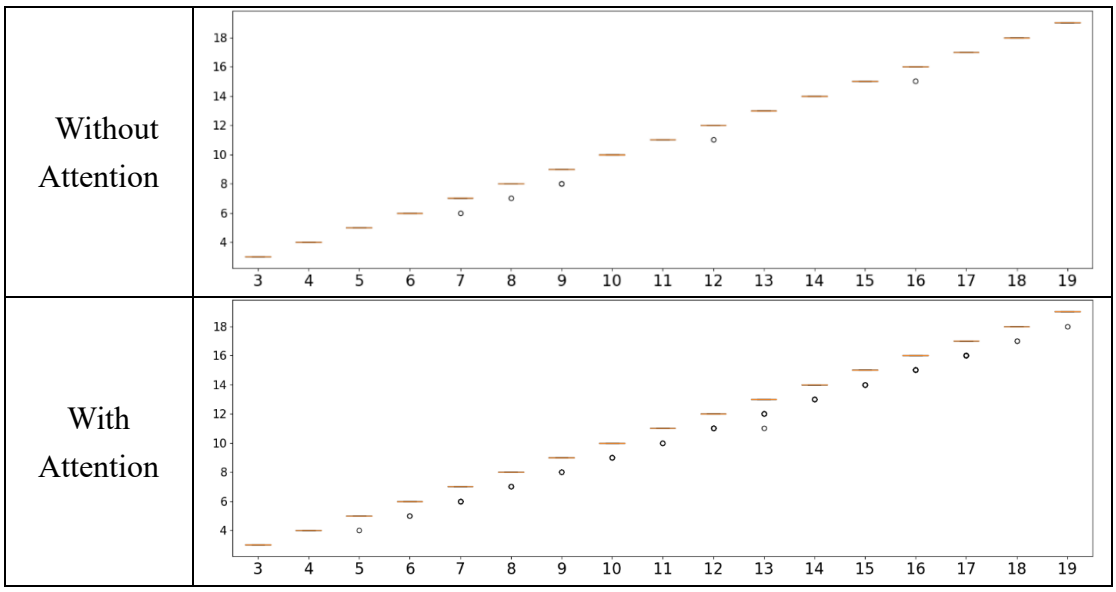


Figure 5-13 The boxplots of model with and without Attention

In this experiment, we observed that there were no significant changes in the repeat ratio and starting point accuracy between the models with and without the Attention mechanism. As for accuracy, the model without Attention performed better

than the model with Attention. The length stability also showed a slight improvement with the inclusion of Attention. However, in terms of diversity, the model with Attention achieved higher values. We consider that while the Attention mechanism helps to maintain diversity in the recommended POI sequences by focusing on important aspects, factors such as the training dataset and the random vector r input to the CGAN still play a role in determining the redundancy and accuracy. Furthermore, incorporating Attention in the Decoder part does not fully exploit its strengths since the input encoded vector η for recommendation is derived from a random vector transformed by the CGAN and doesn't contain complete trajectory information. Considering our goals, we chose the Attention mechanism with better diversity performance as the default option.

In another experiment, we replaced the LSTM modules used in the Autoencoder and CGAN with GRU (Gated Recurrent Unit) and tested the impact of this change on the performance metrics of the entire method. GRU has a simpler structure compared to LSTM, designed to reduce the complexity of LSTM while achieving similar performance. However, LSTM is generally considered to have an advantage in capturing long-term dependencies. Figure 5-14 and Figure 5-15 present the performance metrics of the two architectures.

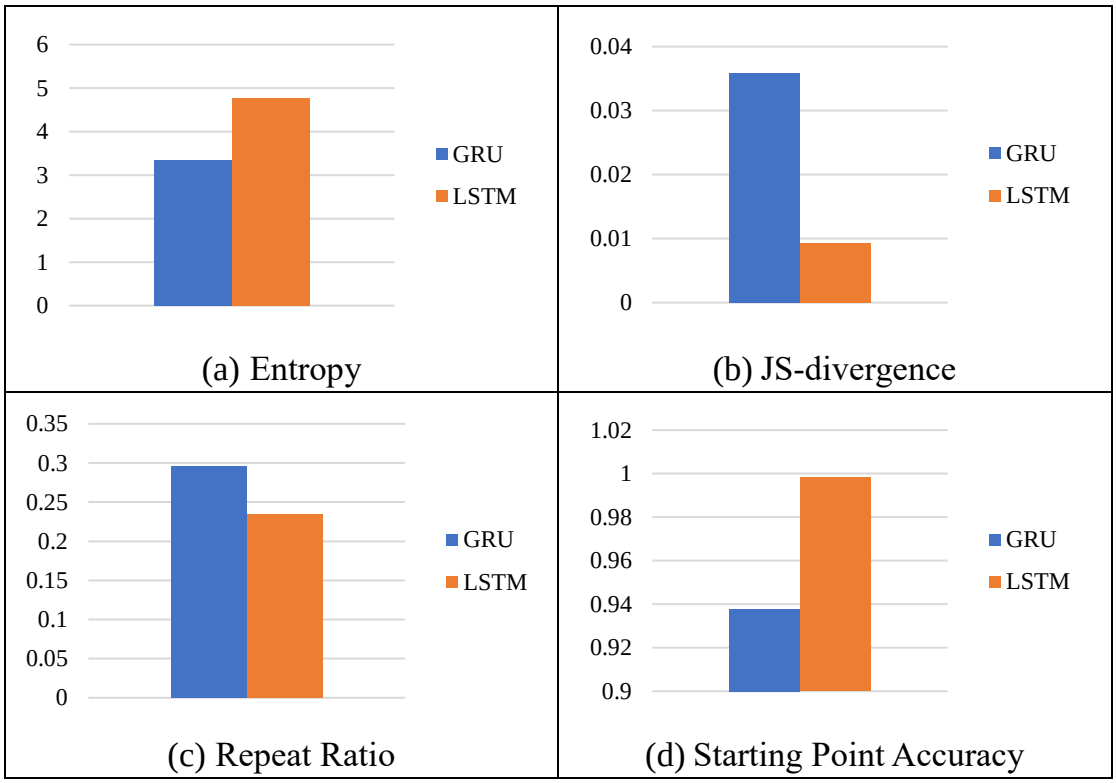


Figure 5-14 The results of different RNN units

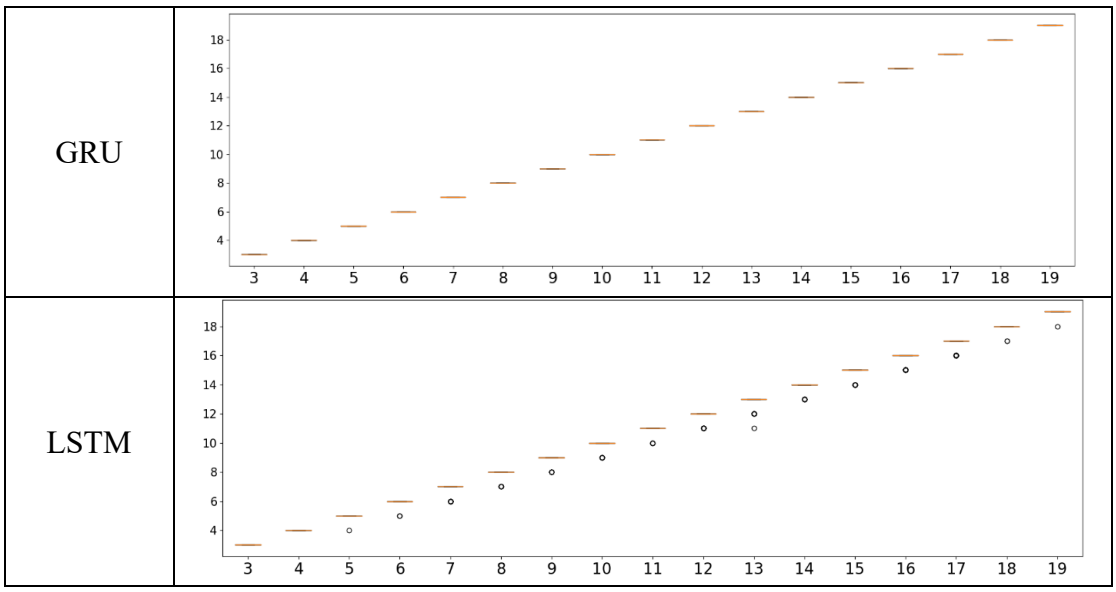


Figure 5-15 The boxplots of different RNN units

From Figure 5-14, it can be observed that LSTM outperforms GRU in all performance metrics. This is largely due to the ability of LSTM to capture long-term dependencies, which can retain long-term information. Similarly, regarding length stability shown in Figure 5-15, GRU demonstrates stable recommendation outcomes,

which may be attributed to its memory capacity. Based on the above analysis, we conclude that using LSTM modules is more suitable for our objectives.

In the final part of the model structure experiment, we wonder the impact of changing the dimensionality of the encoded vector on the performance metrics. Increasing the dimensionality can effectively enhance the model's capacity to store information. However, excessively large dimensions not only increase the number of parameters but may also fail to significantly improve model performance. In this experiment, we trained models with two different encoded vector dimensions: 69 (64 for POI trajectory features + 5 for category trajectory features) and 133 (128 for POI trajectory features + 5 for category trajectory features). The results of these models on the performance metrics are presented in Figure 5-16 and Figure 5-17.

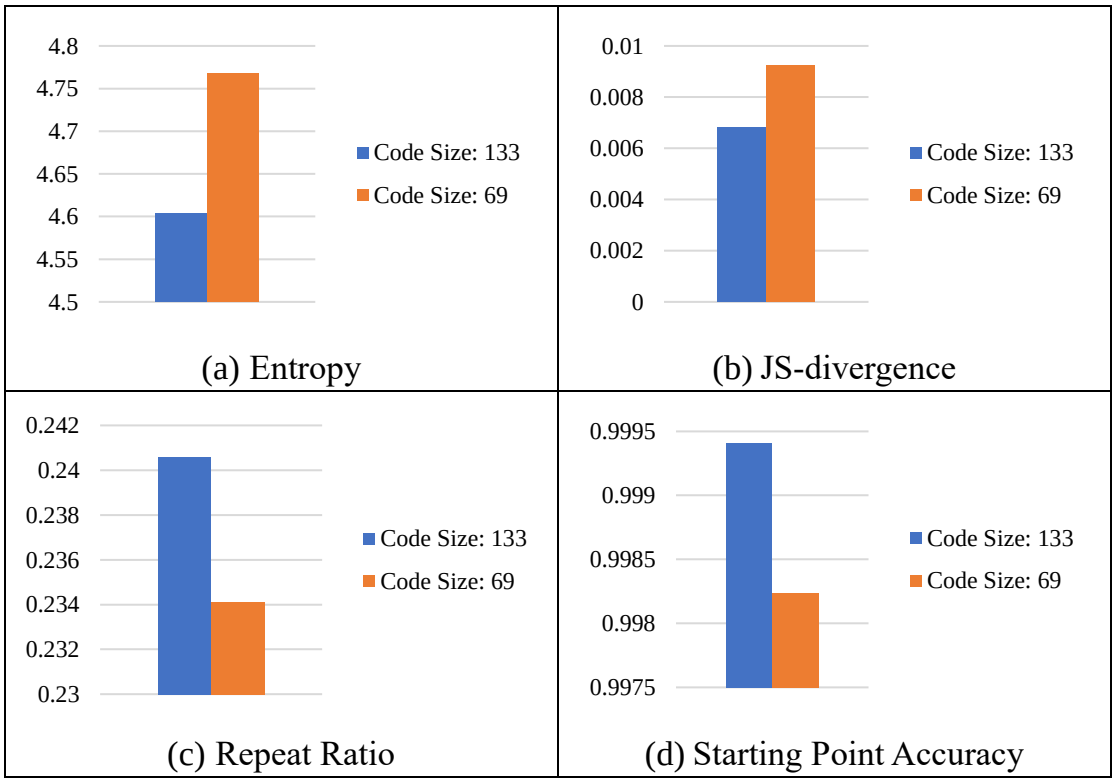


Figure 5-16 The results of different dimension models

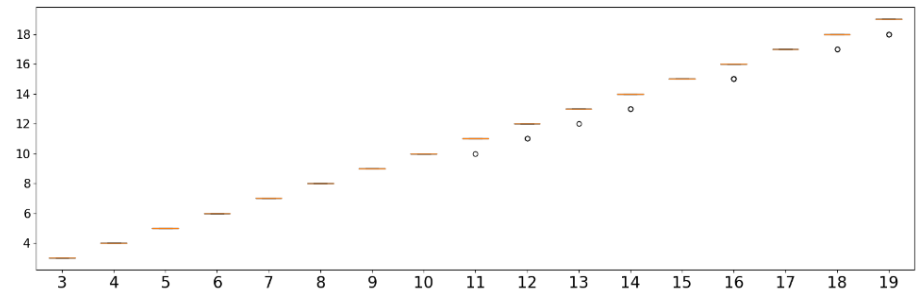
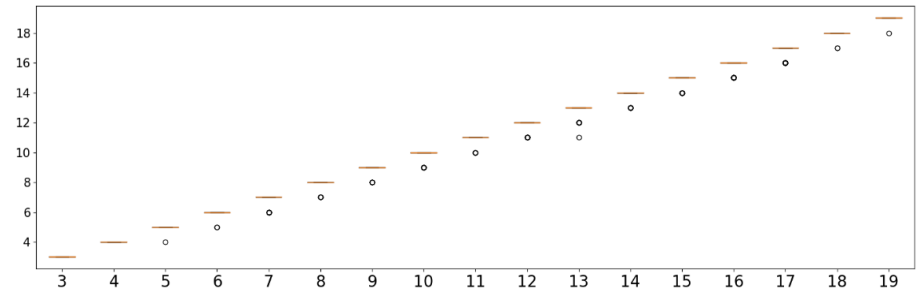
133	
69	

Figure 5-17 The boxplots of different dimension models

From Figure 5-16 and Figure 5-17, we can observe that larger dimensions indeed improve accuracy and enhance length stability. Nevertheless, they lead to a decrease in diversity and an increase in redundancy. In our view, larger dimensions have limited effectiveness and may have reached the performance limit for the model. Increasing the dimensionality not only results in diminishing returns but also increases the number of model parameters, potentially leading to decreased recommendation performance. Therefore, we choose the default parameter setting with a dimensionality of 69.

5.3 External Experiment

In the external experiments, we compared our method with four different literature methods: PERSTOUR, PERSTOUR-L, DeepTrip, and SelfTrip. The first two methods, PERSTOUR and PERSTOUR-L, don't utilize deep learning methods, while the latter two methods, DeepTrip and SelfTrip, employ deep learning methods.

Here, we will provide an overview of these methods:

(1) PERSTOUR [21]: It takes into consideration user preferences and the popularity of POIs and models the recommendation problem based on the orientation problem. It incorporates constraints such as time budget, starting point, and end point, and aims to recommend trips that maximize the score within these constraints.

(2) PERSTOUR-L: It is a variant of PERSTOUR where the time budget constraint is replaced with the trip length.

(3) DeepTrip [14]: It is an end-to-end method that learns human mobility patterns for recommendation and approximates the similarity between queries and trips using adversarial networks.

(4) SelfTrip [12]: It is a self-supervised representation learning method that enhances the interaction between POIs through contrastive learning. It also incorporates a conditional recursive module for modeling and recommending trajectories.

Our external experiments will compare these methods, including ours, in three different topics. The first topic is to test the performance of all methods on different regional datasets. The second topic is to evaluate the performance of recommending different trajectory lengths using our Tokyo Small Dataset. The third topic is to test the recommendation performance for different user groups within the Tokyo Small Dataset. For the second and third topics, we will only compare our method with the deep learning-based methods, namely DeepTrip and SelfTrip. These methods have more complex model structures and representation capabilities, making them suitable for a fair and comparative evaluation. The parameters of our method will be set to the default values discussed in Chapter 5.2, and for the trip group type g, except for the

third theme where it will be separately input for each corresponding user group, all other experiments will use the input of Group All conditions.

5.3.1 Different Datasets

In this external experiment, we utilized the dataset extracted from the Yahoo Flickr Creative Commons 100 Million (YFCC100M) dataset [33] which consists of photos and videos. The extracted dataset is from [21] and includes trips from four cities: Osaka, Glasgow, Edinburgh, and Toronto. Detailed information about the datasets can be found in Table 5-5.

Table 5-5 The detailed information of the datasets

City	**Osaka**	**Glasgow**	**Edinburgh**	**Toronto**
User	450	601	1,454	1,395
Trajectory	1,115	2,227	5,028	6,057
POI	27	27	28	29
Category	4	7	6	6

We applied our proposed method using all the information provided in the dataset and used all the trajectories in the dataset as the test queries to evaluate the performance of all methods. Table 5-6, Table 5-7, and Table 5-8 depict the performance of our method and other methods in terms of diversity, accuracy, redundancy, and stability. As the original dataset doesn't have a balanced number of trajectories for each length, we replaced the length box plots with length accuracy LA in the tables.

Table 5-6 The entropy and JS-divergence results of all methods

City	Osaka		Glasgow		Edinburgh		Toronto	
Metric	ε	D_{JS}	ε	D_{JS}	ε	D_{JS}	ε	D_{JS}
PersTour	**2.798**	0.005	2.766	0.007	2.740	0.003	2.839	0.006
PersTour-L	2.548	0.020	2.702	0.021	2.695	0.007	2.782	0.015
DeepTrip	2.461	0.086	2.648	0.058	2.707	0.151	2.822	0.098
SelfTrip	2.669	0.002	2.741	0.006	2.746	**0.002**	2.868	**0.003**
Our Method	2.724	**0.001**	**2.824**	**0.002**	**2.927**	0.007	**3.137**	0.004

Table 5-7 The repeat ratio and starting point accuracy results of all methods

City	Osaka		Glasgow		Edinburgh		Toronto	
Metric	*RR*	*SA*	*RR*	*SA*	*RR*	*SA*	*RR*	*SA*
PersTour	**0**	1	**0**	1	**0**	1	**0**	1
PersTour-L	**0**	1	**0**	1	**0**	1	**0**	1
DeepTrip	0.277	1	0.161	1	0.331	1	0.299	1
SelfTrip	0.277	1	0.295	1	0.492	1	**0**	1
Our Method	0.404	1	0.339	0.946	0.325	1	0.248	1

Table 5-8 The length accuracy results of all methods

City	Osaka	Glasgow	Edinburgh	Toronto
Metric	*LA*	*LA*	*LA*	*LA*
PersTour	0.426	0.748	0.421	0.603
PersTour-L	1	1	1	1
DeepTrip	1	1	1	1
SelfTrip	1	1	1	1
Our Method	1	0.911	0.970	0.955

Table 5-6 shows the performance in terms of diversity and accuracy. It can be observed that our method achieves almost the best diversity scores in each dataset, except for a slightly lower performance in the Osaka dataset compared to PersTour. In addition, our method exhibits the best accuracy in the Osaka and Glasgow datasets and performs well in the other city datasets. The diversity improvement reached a maximum of 12%, while the accuracy exhibited a remarkable enhancement of up to 98%. This indicates that our method can adapt well to the POI and trajectory data of different datasets, achieving diverse recommendations while maintaining the distribution characteristics of the original data. It can provide users with more realistic trip recommendations. As for redundancy in Table 5-7, PersTour and PersTour-L don't recommend repeated POIs in the recommended trips, while the other deep learning methods have some degree of repetition. This reflects the difference between traditional methods and deep learning methods. Traditional methods can avoid

repetition within predefined rules and constraints, while it is more challenging to directly control the outcomes of recommendations in deep learning methods. Regarding stability, our method relies on the deep learning model to learn and generate recommendations for starting points and lengths. Therefore, occasional errors may occur, but the accuracy is above 90%. PersTour generates recommendations for different lengths based on time budgets, so the stability of this method cannot be evaluated using LA. The other methods directly use the given conditions for recommendation and don't have a comparable stability metric. Therefore, in the subsequent external experiments, we will omit the comparison of this metric.

5.3.2 Different Trajectory Lengths

In this experiment, we will test the performance of deep learning methods under three different conditions: short, medium, and long trajectory lengths. Specifically, we will conduct the experiment on the Tokyo Small Dataset and use trajectory lengths of 3, 11, and 19 to represent the three conditions. We will input the same starting point and trajectory length as the dataset for recommendation and compare the recommended trajectories with the original trajectories as ground truth. Table 5-9 presents the results of all evaluation metrics.

Table 5-9 The results of entropy, JS divergence, and repeat ratio of deep learning methods across different trip lengths

Methods in Different Trajectory Lengths	ε	D_{JS}	RR
DeepTrip for 3 POIs	3.051	0.025	0.768
SelfTrip for 3 POIs	3.553	**0.003**	**0.018**
Our Method for 3 POIs	**3.585**	0.030	0.036
DeepTrip for 11 POIs	2.424	0.487	1
SelfTrip for 11 POIs	2.213	0.105	1
Our Method for 11 POIs	**3.465**	**0.046**	**0.333**
DeepTrip for 19 POIs	1.386	0.255	1
SelfTrip for 19 POIs	1.099	0.817	1
Our Method for 19 POIs	**2.590**	**0.164**	**0**

From Table 5-9, we can observe the performance of the three methods in terms of diversity, accuracy, and redundancy. When it comes to diversity, our proposed method consistently exhibits the most diverse recommended trips across all three lengths, and there is a decreasing trend in diversity as the length increases for all three methods. In terms of accuracy, our method outperformed other deep learning approaches in medium and long lengths, trailing behind DeepTrip and SelfTrip in the short length, indicating that these two methods have better predictive capabilities for shorter trajectories. Besides, all three methods show a decrease in accuracy as the length increases. There is a maximum improvement rate of 135% in diversity and a maximum improvement rate of 90% in accuracy. In the experiment on redundancy, our method also outperformed other deep learning approaches in medium and long lengths, while falling between the other two methods in the short length. DeepTrip and SelfTrip are unable to avoid recommending repetitive POIs in longer lengths. Overall, based on the results of these experiments, we can conclude that our method achieves optimal diversity in recommended trips for any length and exhibits good

performance in accuracy and redundancy.

5.3.3 Different Group Types

In this experiment, we aim to observe whether our method and the deep learning methods can recommend distinct trips among different user groups. In our method, users can input different trip group types corresponding to specific user groups to experience their travel styles. Users can also input trip group types representing all user groups, disregarding specific group preferences, to receive recommendations based on the overall category distribution. Specifically, we divided the Tokyo Small Dataset into three groups: Group 0, Group 1, and Group All (consisting of Group 0 and Group 1). By recommending trajectories for these three groups, we evaluate the recommendation performance across different user groups. The results of this experiment are presented in Table 5-10.

Table 5-10 The results of entropy, JS divergence, and repeat ratio of deep learning methods across different groups

Methods in Different Group	ε	D_{JS}	RR
DeepTrip in Group0	3.850	0.273	0.823
SelfTrip in Group0	3.777	**0.005**	0.708
Our Method in Group0	**4.400**	0.013	**0.099**
DeepTrip in Group1	2.445	0.204	0.818
SelfTrip in Group1	2.672	0.080	0.727
Our Method in Group1	**3.509**	**0.016**	**0.136**
DeepTrip in Group All	3.949	0.265	0.822
SelfTrip in Group All	3.872	**0.006**	0.710
Our Method in Group All	**4.455**	0.011	**0.098**

From the experimental results, we can conclude that even when different user groups are used as conditions, our method exhibits the best diversity and redundancy. The accuracy is comparable to the best-performing method, and notably, our method

performs better than others in Group 1. Specifically, our method can recommend category distributions that are similar to the given trip group type conditions, and it shows the most similar category distribution and best matches user preferences in Group All. However, other methods, which aim for prediction as the recommendation objective, can achieve category distributions that are closer to the ground truth. Our method, on the other hand, sacrifices a slight decrease in accuracy due to the input of random vectors, but in return, it achieves the best diversity and recommends more non-repetitive POIs in the trips. The experiment demonstrated significant improvements in diversity, with a maximum improvement rate of 43%, and in accuracy, showing a remarkable improvement rate of up to 95%.

5.4 The Results of Trip Recommendations

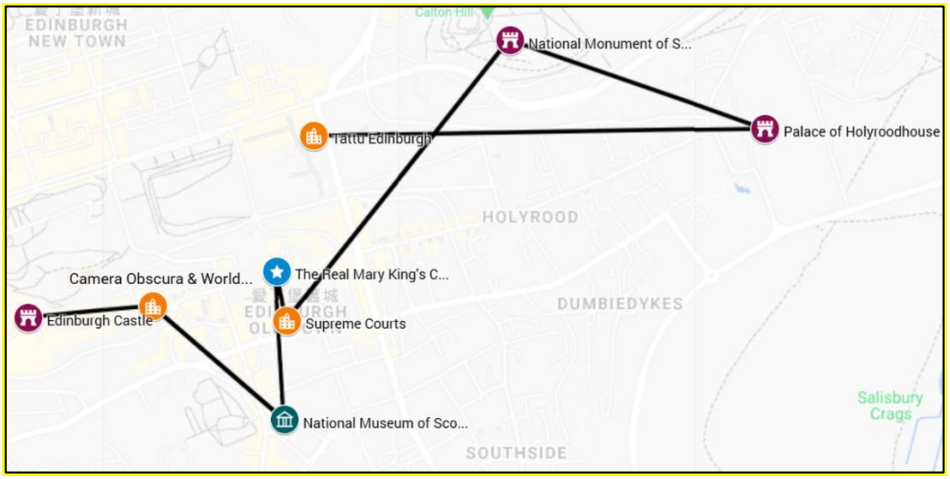
In this chapter, we compare the practical outcomes of trip recommendations using our method and other existing methods. Firstly, we select two cities, Edinburgh and Toronto, from the datasets mentioned in Chapter 5.3.1. For each of these cities, we apply the deep learning methods, DeepTrip, SelfTrip and our method for trip recommendations. The results of these recommendations are then visualized on maps, the legend in the bottom right corner illustrates the POI categories along with the quantity of POIs in each category. In more detail, we individually select one short and one long trajectory from each of these two cities. Subsequently, we employ both our proposed method and other methods to provide trip recommendations for these chosen trajectories. After that, we analyze the outcomes of these recommendations and proceed to compare the differences in performance between the methods.

5.4.1 Recommended Results in Edinburgh

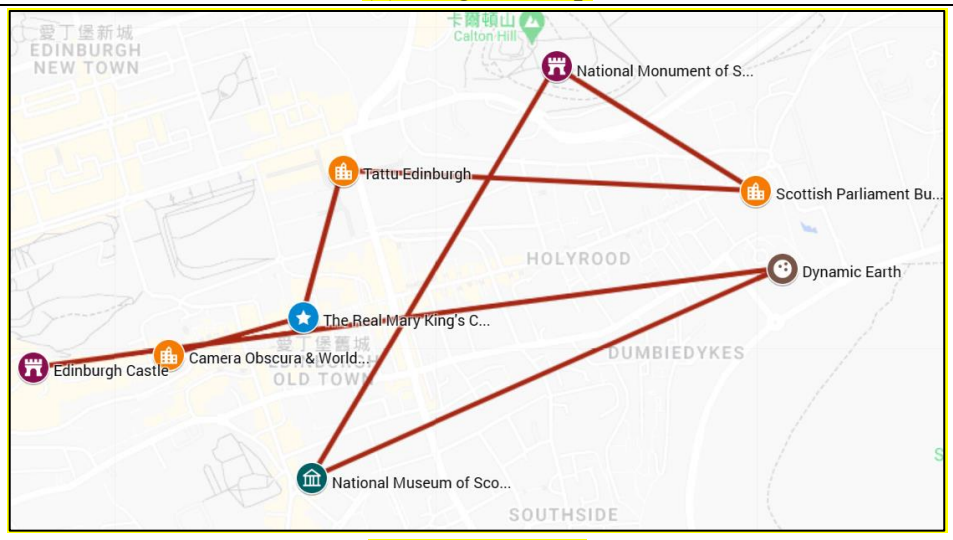
Figure 5-18 The recommendation results of a short trip in Edinburgh

Figure 5-18 illustrates the outcomes of short trip recommendations in Edinburgh, with Calton Hill as the starting point. From a predictive standpoint, both DeepTrip and SelfTrip failed to accurately predict the entire original trajectory. Only some of the POIs were predicted correctly. In terms of diversity, DeepTrip recommended a POI of a different category, "The Real Mary King's Close," and a POI from the original trajectory, "The Jazz Bar." SelfTrip recommended a POI of the same category, "Camera Obscura & World of Illusions," and a POI from the original trajectory, "Supreme Courts." Our method recommended a POI of a different category, "The

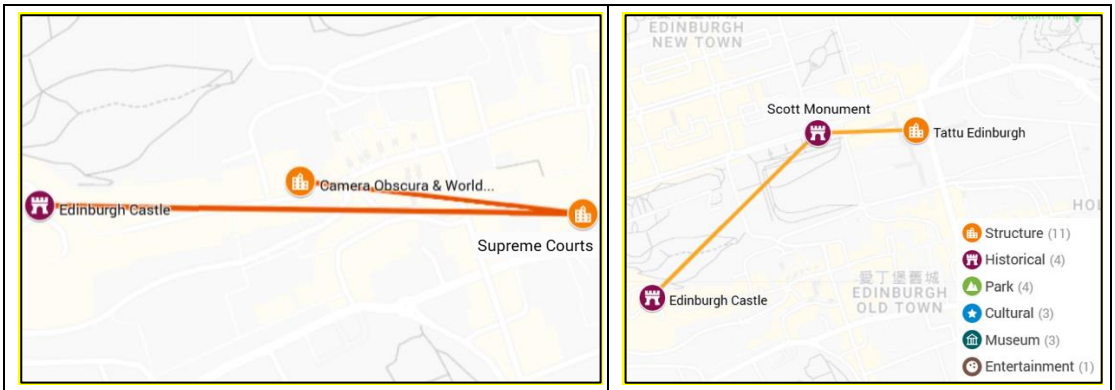
Real Mary King's Close," and a POI of the same category but entirely different from the original trajectory, "Allan Ramsay Monument." This observation highlights our method's superiority in terms of diversity on two fronts: one involving the recommendation of POIs from different categories than those previously encountered and another involving the recommendation of POIs within the same category but distinct from the original trajectory. In practical application, our method indeed excels in suggesting trips enriched with a variety of categories and diverse POIs.



(a) Original Trip



(b) Our Method



(c) DeepTrip (d) SelfTrip

Figure 5-19 The recommendation results of a long trip in Edinburgh

Figure 5-19 shows the outcomes of recommendations for long trips, with Edinburgh Castle as the starting point. The results align with the analysis in Chapter 5.3.2. Both DeepTrip and SelfTrip can't perform well in recommending long trajectories. While these methods can suggest trips of specified lengths, they often generate recommendations with a high occurrence of duplicate POIs or end symbols, resulting in monotonous and short trajectory suggestions that are not practical in real-world scenarios. In contrast, our method excels at recommending trips of the specified length. However, due to the limited number of POIs in the dataset, our recommendations sometimes feature POIs that are similar to those in the original trajectory. From the results of our method's recommendations, it is evident that despite the relatively small number of POIs available, our approach can still provide recommendations that align with the given length, are diverse, and feature a different sequence of POIs. At the category level, the recommended trajectory includes a POI of the Entertainment category, "Dynamic Earth," which was not present in the original trajectory. At the POI level, a POI of the Structure category, "Scottish Parliament Building," was recommended, and it differs from the POIs in the original trajectory. In this scenario, even though the recommendations may not entirely avoid previously visited POIs, they still serve as valuable references for trip planning.

5.4.2 Recommended Results in Toronto

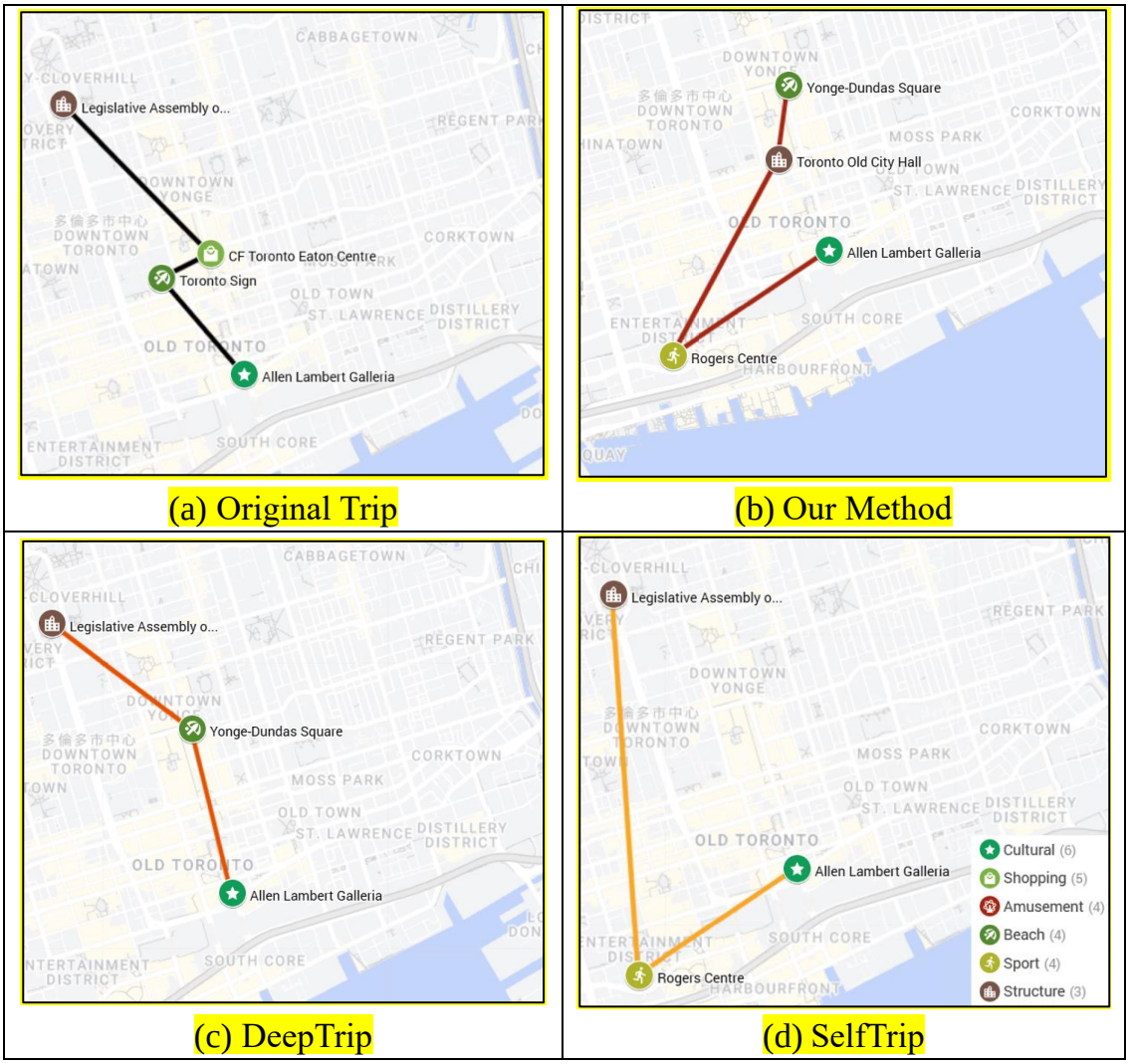


Figure 5-20 The recommendation results of a short trip in Toronto

From Figure 5-20, we can observe the recommendation results of the three methods for short trips in the Toronto dataset, starting from Allen Lambert Galleria. Our method successfully recommends a 4-POI trip based on the specified conditions. The other two deep learning methods also recommend 4 POIs, but they include repeated POIs and end symbols in their recommendations, resulting in a less clear composition of the recommended trip. In terms of prediction accuracy, DeepTrip and SelfTrip only predict the correct starting and ending points, with the other POIs not appearing in the original trajectory. However, in the diversity aspect, our method

effectively recommends 4 distinct POIs of different categories. Except for the starting point, none of the recommended POIs match those in the original trajectory. Furthermore, within the recommended itinerary, the POIs of the same category differ from those in the original trajectory. For instance, Toronto Old City Hall and Yonge-Dundas Square showcase this variation. Moreover, a Sports category POI, Rogers Centre, is introduced in the recommendation that was not present in the original trajectory. These results demonstrate that our method can also perform well for diverse recommendations across different datasets.

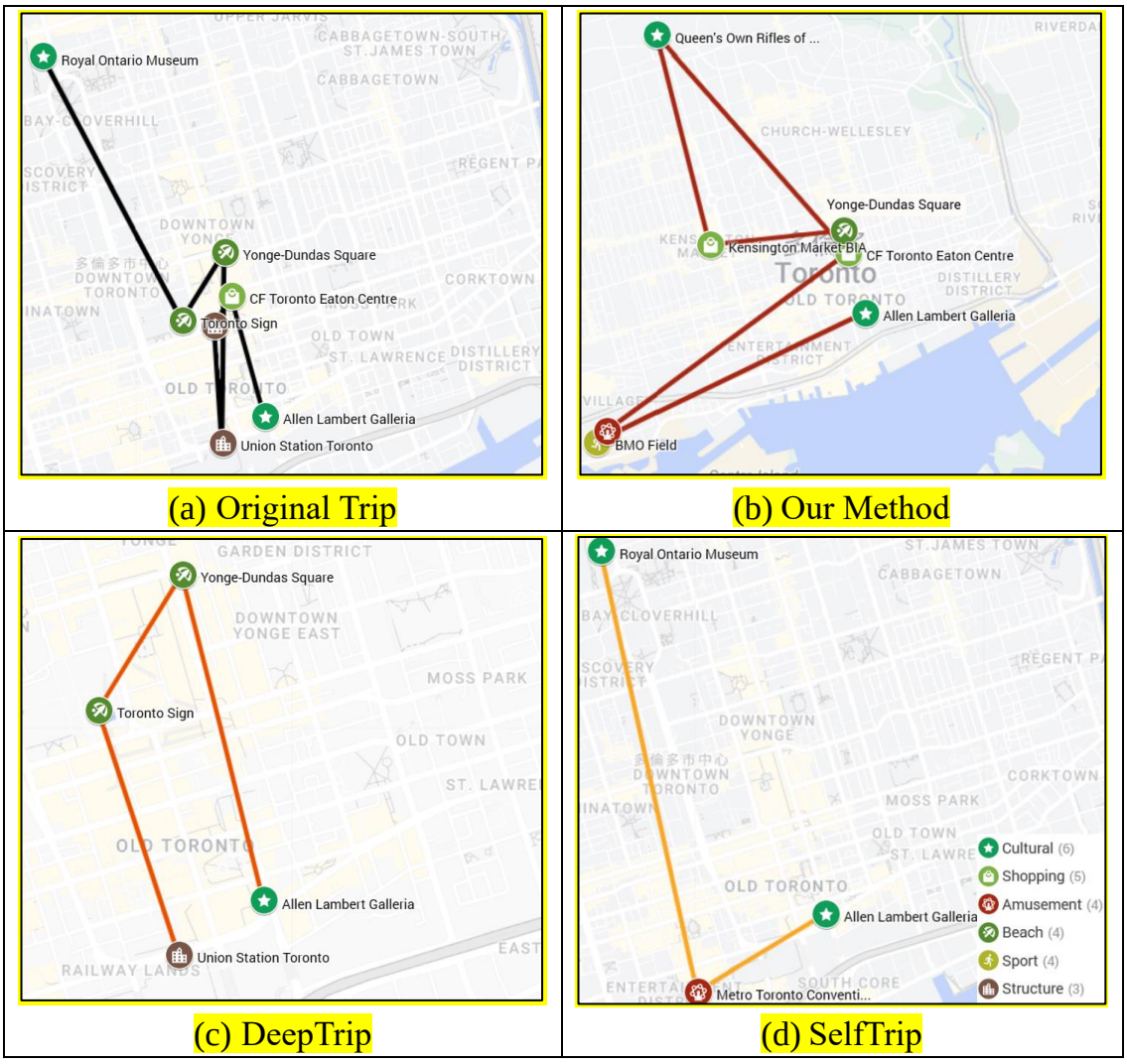


Figure 5-21 The recommendation results of a long trip in Toronto

In the context of recommending long trips in Toronto, Figure 5-21 illustrates the actual recommendation outcomes with the starting point still being Allen Lambert Galleria. In terms of prediction accuracy as the objective, both DeepTrip and SelfTrip perform poorly in recommending long trips. While the recommended length aligns, the majority of POIs are either repeated or involve end symbols, resulting in a rather monotonous presentation. However, DeepTrip's recommended trip includes POIs that appear in the original trajectory, showing a relatively better performance compared to SelfTrip. Nonetheless, both of these methods struggle to provide diverse itineraries effectively. In contrast, our method's recommended trip doesn't contain repeated POIs and covers almost all categories of the original trajectory. Only two POIs, Yonge-Dundas Square and CF Toronto Eaton Centre, from the original trajectory are included in the recommended trip plan. In this recommended trip, there are two additional categories, Sport and Amusement, which are not present in the original trajectory. This serves to offer users a distinct travel experience. Furthermore, the other recommended POIs also differ from those in the original trajectory. Not only does our method ensure category diversity, but it also maintains a certain level of variation in terms of the individual POIs.

Based on the actual recommendation for the two cities and two trip lengths mentioned above, most methods are successful in recommending the specified number of POIs for short trips. Our method excels in short trip recommendations by providing diversified results. It not only introduces previously unencountered POIs but also introduces new categories, both on the category and POI levels. Regarding the recommendations for long trips, DeepTrip and SelfTrip are hard to recommend POIs for the specified length effectively. In contrast, our approach can generate recommended trips that match the given length. While there might be instances of

recommending POIs that overlap with the original trajectory, this can be mitigated with a dataset containing a more extensive set of POIs. This is evidenced by the Toronto long trip recommendation, which includes several non-original POIs, demonstrating diversity in its characteristics. This indicates that our method consistently performs well across different datasets. In summary, our method is designed to cater to real-world user travel needs. It generates recommended trips based on given conditions, serving as either direct recommendations or references for travel planning. Additionally, the introduction of random vectors ensures that each recommended trip is unique. As indicated through the analysis of recommendation, category, and POI diversity, our method excels in offering diverse experiences in all these dimensions. In this chapter's discussion, we have indeed achieved our set goals. Through our method, we have successfully provided diverse travel experiences that differ from past experiences in the real world. This approach not only diversifies the travel trip but also serves to disperse excessive crowds, allowing users to explore a broader range of tourist attractions.

Chapter 6 Conclusions and Future Work

In this thesis, we propose a trip recommendation method based on Autoencoder and CGAN to address the issue of diversity in trip recommendations. Our method generates diverse trip trajectories based on user query conditions. First, to tackle the problem of insufficient trajectory data, we design a data augmentation method that preserves the category distribution and controls the trajectory length. Next, we employ an Autoencoder architecture to extract features from the trajectories. The Encoder is trained to encode the trajectories into low-dimensional vectors, and the Decoder reconstructs the trajectories from these vectors. Furthermore, we utilize CGAN to generate low-dimensional vectors that are similar to the encoded vectors. CGAN considers various conditions such as trajectory length and trip group type to generate corresponding low-dimensional features. During the training of CGAN, we introduce negative samples based on trajectory length conditions to enhance the stability and meet length constraints in the generated vectors. Finally, we can generate low-dimensional vectors by inputting random vectors and query conditions, and then convert them back to so-called diverse recommendation trajectories through the Decoder.

In our internal experiments, we adjust multiple parameters and settings to maximize the diversity of recommendations, and other metrics are also above the standard. We then conduct external experiments using the best parameter settings to compare our method with other literature methods. Across different datasets, our method outperforms almost all methods in terms of diversity, with improvements of up to 12% observed. When comparing performance at different trajectory lengths, our method has better results in diversity metric, achieving up to a 135%

improvement in the longest trajectory length. This case highlights that our method excels in recommending diverse trip trajectories at any length and suggests that the compared methods are less suitable for recommending longer trajectories. In the comparison of different user groups, our method consistently exhibits the highest diversity, indicating its ability to provide diverse recommendations across different user groups. Especially in the experiment involving Group 1, there is a 43% improvement in diversity. These results indicate that the recommended trajectories generated by our approach align with the category distribution of different user groups. They also signify that the recommendations conform to their preferences and offer diverse suggestions based on their difference.

In the future, there are several aspects in which we can further enhance our trip recommendation research. First, the datasets used in this study have a certain age, and the travel patterns may have changed due to the pandemic. Future efforts could involve collecting anonymous travel data or utilizing reference itineraries from travel agencies as the basis for new datasets. After the dataset is updated, we can obtain a more enriched travel experience and data. Additionally, we can consider augmenting the original trajectory data with additional trajectory information from various perspectives to obtain more realistic augmented data. With these enriched augmented data, we can incorporate additional features for model training, such as spatial features, temporal features, trip purposes, and visual features, and extend the scope of recommendations to include route planning, transportation mode arrangement, filtering out non-operating businesses, and more applications. Furthermore, these attribute features, once extracted into low-dimensional vectors by the Encoder, will possess deeper meanings and utility value. They can be used to enhance subtle differences in clustering, recommend similar trips in different regions, analyze human

mobility behavior patterns, and more. For model architectures, there are various models available for feature extraction, such as Graph Neural Networks (GNNs), which establish relationships between information using nodes and edges. In recent years, the development of generative network deep learning has made considerable breakthroughs. In addition to GAN, many more powerful generative networks have been applied to image generation, such as diffusion models, and can also be generated according to input conditions. Applying these models to sequence generation and their ability to generate output based on input conditions hold great research potential. When it comes to input conditions, our study only considers the starting point, trajectory length, and trip group type as conditions. We hope to incorporate additional conditions and variables to make trip recommendations more flexible and practical. In the end, the essence of this research lies in the recommendation of trip trajectories, with the key factor being the user's experience. In addition to evaluating the results using objective experimental metrics, it may be beneficial to include user experience in specific experimental areas as an evaluation metric, weighted according to the importance of each metric, to provide a more comprehensive and objective comparison.

References

[1] Abbasi-Moud, Z., Vahdat-Nejad, H., & Sadri, J. (2021). Tourism Recommendation System Based on Semantic Clustering and Sentiment Analysis. *Expert Systems with Applications*, *167*, p. 114324.

[2] Chen, L., Cao, J., Chen, H., Liang, W., Tao, H., & Zhu, G. (2021). Attentive Multi-Task Learning for Group Itinerary Recommendation. *Knowledge and Information Systems*, *63*(7), pp. 1687-1716.

[3] Cepeda-Pacheco, J. C., & Domingo, M. C. (2022). Deep Learning and Internet Of Things for Tourist Attraction Recommendations in Smart Cities. *Neural Computing and Applications*, *34*(10), pp. 7691-7709.

[4] Chen, L., Zhang, L., Cao, S., Wu, Z., & Cao, J. (2020). Personalized Itinerary Recommendation: Deep and Collaborative Learning with Textual Information. *Expert Systems with Applications*, *144*, p. 113070.

[5] Duan, Z., Gao, Y., Feng, J., Zhang, X., & Wang, J. (2020). Personalized Tourism Route Recommendation Based on User's Active Interests. In *2020 21st IEEE International Conference on Mobile Data Management (MDM)*, pp. 729-734.

[6] Dietz, L. W., Sen, A., Roy, R., & Wörndl, W. (2020). Mining Trips from Location-Based Social Networks for Clustering Travelers and Destinations. *Information Technology & Tourism*, *22*, pp. 131-166.

[7] Figueredo, M., Ribeiro, J., Cacho, N., Thome, A., Cacho, A., Lopes, F., & Araujo, V. (2018). From Photos to Travel Itinerary: A Tourism Recommender System for Smart Tourism Destination. In *2018 IEEE Fourth International Conference on Big Data Computing Service and Applications (BigDataService)*, pp. 85-92.

[8] Gao, R., Li, J., Li, X., Song, C., Chang, J., Liu, D., & Wang, C. (2018). STSCR: Exploring Spatial-Temporal Sequential Influence and Social Information for Location Recommendation. *Neurocomputing*, *319*, pp. 118-133.

[9] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2020). Generative Adversarial Networks. *Communications of the ACM*, *63*(11), pp. 139-144.

[10] Gu, J., Song, C., Jiang, W., Wang, X., & Liu, M. (2020). Enhancing Personalized Trip Recommendation with Attractive Routes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, *34*(01), pp. 662-669.

[11] Gao, Q., Wang, W., Huang, L., Yang, X., Li, T., & Fujita, H. (2023). Dual-Grained Human Mobility Learning for Location-Aware Trip Recommendation with Spatial–Temporal Graph Knowledge Fusion. *Information Fusion*, *92*, pp. 46-63.

[12] Gao, Q., Wang, W., Zhang, K., Yang, X., Miao, C., & Li, T. (2022). Self-Supervised

Representation Learning for Trip Recommendation. *Knowledge-Based Systems*, *247*, p. 108791.

[13] Gao, Q., Zhang, F., Yao, F., Li, A., Mei, L., & Zhou, F. (2020). Adversarial Mobility Learning for Human Trajectory Classification. *IEEE Access*, *8*, pp. 20563-20576.

[14] Gao, Q., Zhou, F., Zhang, K., Zhang, F., & Trajcevski, G. (2023). Adversarial Human Trajectory Learning for Trip Recommendation. *IEEE transactions on neural networks and learning systems*, *34*(4), pp. 1764-1776.

[15] Han, S., Liu, C., Chen, K., Gui, D., & Du, Q. (2021). A Tourist Attraction Recommendation Model Fusing Spatial, Temporal, and Visual Embeddings for Flickr-Geotagged Photos. *ISPRS International Journal of Geo-Information*, *10*(1), p. 20.

[16] Huang, L., Ma, Y., Wang, S., & Liu, Y. (2019). An Attention-Based Spatiotemporal LSTM Network for Next POI Recommendation. *IEEE Transactions on Services Computing*, *14*(6), pp. 1585-1597.

[17] He, J., Qi, J., & Ramamohanarao, K. (2019). A Joint Context-Aware Embedding for Trip Recommendations. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pp. 292-303.

[18] Huang, F., Xu, J., & Weng, J. (2020). Multi-Task Travel Route Planning with A Flexible Deep Learning Framework. *IEEE Transactions on Intelligent Transportation Systems*, *22*(7), pp. 3907-3918.

[19] Jiang, S., Qian, X., Mei, T., & Fu, Y. (2016). Personalized Travel Sequence Recommendation on Multi-Source Big Social Media. *IEEE Transactions on Big Data*, *2*(1), pp. 43-56.

[20] Jiang, L., Zhou, J., Xu, T., Li, Y., Chen, H., & Dou, D. (2022). Time-Aware Neural Trip Planning Reinforced by Human Mobility. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pp. 1-8.

[21] Lim, K. H., Chan, J., Leckie, C., & Karunasekera, S. (2015). Personalized Tour Recommendation Based on User Interests and Points of Interest Visit Durations. In *Twenty-Fourth International Joint Conference on Artificial Intelligence*.

[22] Lim, K. H., Chan, J., Leckie, C., & Karunasekera, S. (2018). Personalized Trip Recommendation for Tourists Based on User Interests, Points of Interest Visit Durations and Visit Recency. *Knowledge and Information Systems*, *54*, pp. 375-406.

[23] Luan, W., Liu, G., Jiang, C., & Zhou, M. (2018). MPTR: A Maximal-Marginal-Relevance-Based Personalized Trip Recommendation Method. *IEEE Transactions on Intelligent Transportation Systems*, *19*(11), pp. 3461-3474.

[24] Liu, Y., Wei, W., Sun, A., & Miao, C. (2014). Exploiting Geographical Neighborhood Characteristics for Location Recommendation. In *Proceedings of the 23rd ACM international conference on conference on information and knowledge management*, pp.

739-748).

[25] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, *1*(14), pp. 281-297.

[26] Memon, I., Chen, L., Majid, A., Lv, M., Hussain, I., & Chen, G. (2015). Travel Recommendation Using Geo-Tagged Photos in Social Media for Tourist. *Wireless Personal Communications*, *80*, pp. 1347-1362.

[27] Noorian, A., Harounabadi, A., & Ravanmehr, R. (2022). A Novel Sequence-Aware Personalized Recommendation System Based on Multidimensional Information. *Expert Systems with Applications*, *202*, p. 117079.

[28] Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, *20*, pp. 53-65.

[29] Sun, X., Huang, Z., Peng, X., Chen, Y., & Liu, Y. (2019). Building A Model-Based Personalised Recommendation Approach for Tourist Attractions from Geotagged Social Media Data. *International Journal of Digital Earth*, *12*(6), pp. 661-678.

[30] Sun, C. Y., & Lee, A. J. (2017). Tour Recommendations by Mining Photo Sharing Social Media. *Decision Support Systems*, *101*, pp. 28-39.

[31] Sarkar, J. L., & Majumder, A. (2021). A New Point-of-Interest Approach Based on Multi-Itinerary Recommendation Engine. *Expert Systems with Applications*, *181*, p. 115026.

[32] Sarkar, J. L., & Majumder, A. (2022). gtour: Multiple Itinerary Recommendation Engine for Group of Tourists. *Expert Systems with Applications*, *191*, p. 116190.

[33] Thomee, B., Shamma, D. A., Friedland, G., Elizalde, B., Ni, K., Poland, D., Borth, D., & Li, L. J. (2016). YFCC100M: The New Data in Multimedia Research. *Communications of the ACM*, *59*(2), pp. 64-73.

[34] Wen, Y. T., Yeo, J., Peng, W. C., & Hwang, S. W. (2017). Efficient Keyword-Aware Representative Travel Route Recommendation. *IEEE Transactions on Knowledge and Data Engineering*, *29*(8), pp. 1639-1652.

[35] Wang, X., Zhao, Y. L., Nie, L., Gao, Y., Nie, W., Zha, Z. J., & Chua, T. S. (2014). Semantic-Based Location Recommendation with Multimodal Venue Semantics. *IEEE Transactions on Multimedia*, *17*(3), pp. 409-419.

[36] Yochum, P., Chang, L., Gu, T., & Zhu, M. (2020). An Adaptive Genetic Algorithm for Personalized Itinerary Planning. *IEEE Access*, *8*, pp. 88147-88157.

[37] Yang, D., Zhang, D., Zheng, V. W., & Yu, Z. (2014). Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *45*(1), pp. 129-142.

[38] Zhang, K., Chen, Y., & Li, C. (2019). Discovering The Tourists' Behaviors and

Perceptions in A Tourism Destination by Analyzing Photos' Visual Content with A Computer Deep Learning Model: The Case of Beijing. *Tourism Management*, *75*, pp. 595-608.

[39] Zhao, J., Fang, J., Chao, P., Ning, B., & Zhang, R. (2023). GC-TripRec: Graph Contextualized Generative Network with Adversarial Learning for Trip Recommendation. *World Wide Web*, pp. 1-20.

[40] Zhang, C., Liang, H., & Wang, K. (2016). Trip Recommendation Meets Real-World Constraints: POI Availability, Diversity, and Traveling Time Uncertainty. *ACM Transactions on Information Systems (TOIS)*, *35*(1), pp. 1-28.

[41] Zhou, F., Wu, H., Trajcevski, G., Khokhar, A., & Zhang, K. (2020). Semi-Supervised Trajectory Understanding with POI Attention for End-to-End Trip Recommendation. *ACM Transactions on Spatial Algorithms and Systems (TSAS)*, *6*(2), pp. 1-25.

[42] Zhou, F., Wang, P., Xu, X., Tai, W., & Trajcevski, G. (2021). Contrastive Trajectory Learning for Tour Recommendation. *ACM Transactions on Intelligent Systems and Technology (TIST)*, *13*(1), pp. 1-25.

[43] Zhao, P., Xu, C., Liu, Y., Sheng, V. S., Zheng, K., Xiong, H., & Zhou, X. (2021). Photo2Trip: Exploiting Visual Contents in Geo-Tagged Photos for Personalized Tour Recommendation. *IEEE Transactions on Knowledge and Data Engineering*, *33*(4), pp. 1708-1721.

[44] Zhao, J., Zhang, L., Ye, J., & Xu, C. (2021). MDLF: A Multi-View-Based Deep Learning Framework for Individual Trip Destination Prediction in Public Transportation Systems. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), pp. 13316-13329.

[45] 報復性出國旅遊，專家：可能只會維持一年 https://technews.tw/2023/07/10/tourism-boom/ (20230710)

[46] How Long Will The Travel Boom Last? https://www.economist.com/business/2023/06/15/how-long-will-the-travel-boom-last (20230615)